```
cloudra/tools/core/xtb_path_runner.py:34    **路径精修必须跑到引擎自己判收敛**（本机 xtb 6.7.1 实测，逐字证据与可重跑脚本在
cloudra/tools/core/xtb_path_runner.py:35    ``docs/evidence/ts_guess_mtd_artifact/``）：xTB 的路径是**迭代精修**的，而它的内建上限只有
cloudra/tools/core/xtb_path_runner.py:36    3 轮，3 轮后会打印 ``terminated because max. # cycles reached`` —— 那不是收敛，是**被截断**。
cloudra/tools/core/xtb_path_runner.py:37    截断路径的最高帧落在反应物自己的盆地里（实测：最高帧离起点只有 21°，相对能 44.865 kcal/mol，
cloudra/tools/core/xtb_path_runner.py:38    拿去做 ORCA ``OptTS`` 跑 56 轮不收敛、一路滑回极小值）。把上限放开后，同一个端点对在第 31 轮
cloudra/tools/core/xtb_path_runner.py:39    由引擎自己判收敛（``terminated because grad at TS <threshold``），barrier 降到 18.299 kcal/mol，
cloudra/tools/core/xtb_path_runner.py:40    最高帧落在扭转极大上，``OptTS`` 9 轮即收敛到一阶鞍点。因此本模块：
cloudra/tools/core/xtb_path_runner.py:41
cloudra/tools/core/xtb_path_runner.py:42    - **总是**写一份 ``$path`` 输入并把 ``nrun`` 上限显式传给引擎（内建默认 3 轮不能用）；
cloudra/tools/core/xtb_path_runner.py:43    - 把这两句终止语**分开分类**：``converged`` / ``cycles_exhausted``；
cloudra/tools/core/xtb_path_runner.py:44    - 触到上限一律报 ``path_not_converged`` 且 ``usable_as_guess=False``，绝不把截断路径当可用初猜。
cloudra/tools/core/xtb_path_runner.py:45    """
cloudra/tools/core/xtb_path_runner.py:46
cloudra/tools/core/xtb_path_runner.py:47    from __future__ import annotations
cloudra/tools/core/xtb_path_runner.py:48
cloudra/tools/core/xtb_path_runner.py:49    import os
cloudra/tools/core/xtb_path_runner.py:50    import subprocess
cloudra/tools/core/xtb_path_runner.py:51    from typing import Literal, Optional
cloudra/tools/core/xtb_path_runner.py:52
cloudra/tools/core/xtb_path_runner.py:53    from pydantic import BaseModel, Field
cloudra/tools/core/xtb_path_runner.py:54
cloudra/tools/core/xtb_path_runner.py:55    from cloudra.persistence.fs_utils import atomic_write_text
cloudra/tools/core/xtb_path_runner.py:56    from cloudra.tools.core.run_dirs import (
cloudra/tools/core/xtb_path_runner.py:57        capture_versions,
cloudra/tools/core/xtb_path_runner.py:58        list_artifact_files,
cloudra/tools/core/xtb_path_runner.py:59        next_run_dir,
cloudra/tools/core/xtb_path_runner.py:60        terminate_process_group,
cloudra/tools/core/xtb_path_runner.py:61    )
cloudra/tools/core/xtb_path_runner.py:62
cloudra/tools/core/xtb_path_runner.py:63    PATH_XYZ_NAME = "xtbpath.xyz"
cloudra/tools/core/xtb_path_runner.py:64    TS_XYZ_NAME = "xtbpath_ts.xyz"
cloudra/tools/core/xtb_path_runner.py:65    RUNS_DIR_NAME = "xtb_path_runs"
cloudra/tools/core/xtb_path_runner.py:66    #: 引擎逐迭代写下的中间帧命名（``xtbpath_1.xyz`` … ``xtbpath_N.xyz``）。只在引擎没走到收敛、
cloudra/tools/core/xtb_path_runner.py:67    #: 没写出最终 ``xtbpath.xyz`` 时才存在——它们是**采样现场，不是初猜**（GH #39）。
cloudra/tools/core/xtb_path_runner.py:68    PARTIAL_FRAME_PREFIX = "xtbpath_"
cloudra/tools/core/xtb_path_runner.py:69    PARTIAL_FRAME_SUFFIX = ".xyz"
cloudra/tools/core/xtb_path_runner.py:70    MAX_EVIDENCE_LINES = 20
cloudra/tools/core/xtb_path_runner.py:71
cloudra/tools/core/xtb_path_runner.py:72    # --- xtb 6.7.1 的逐字标记（精确比对，不做模糊匹配；本模块不含正则） ---
cloudra/tools/core/xtb_path_runner.py:73    MARKER_NORMAL_TERMINATION = "normal termination of xtb"
cloudra/tools/core/xtb_path_runner.py:74    MARKER_FINISHED_RUN = " * finished run on"
cloudra/tools/core/xtb_path_runner.py:75    MARKER_PATH_FINDER = " path finder:"
cloudra/tools/core/xtb_path_runner.py:76    #: 路径精修的两句终止语。前者是引擎判的收敛，后者是**触到轮数上限被截断**，二者绝不等价。
cloudra/tools/core/xtb_path_runner.py:77    MARKER_PATH_CONVERGED = "terminated because grad at TS <threshold"
cloudra/tools/core/xtb_path_runner.py:78    MARKER_PATH_CYCLES_EXHAUSTED = "terminated because max. # cycles reached"
cloudra/tools/core/xtb_path_runner.py:79    #: 每轮精修打印的 barrier（第一列是空格对齐，逐字保留）
cloudra/tools/core/xtb_path_runner.py:80    MARKER_FORWARD_BARRIER = "forward  barrier (kcal)"
cloudra/tools/core/xtb_path_runner.py:81    MARKER_BACKWARD_BARRIER = "backward barrier (kcal)"
cloudra/tools/core/xtb_path_runner.py:82
cloudra/tools/core/xtb_path_runner.py:83    #: 路径精修的轮数上限（``$path nrun``）。这**不是**收敛判据，只是机时上限：收敛由引擎自己判
cloudra/tools/core/xtb_path_runner.py:84    #: （见模块 docstring 的实测）。默认给足，让引擎跑到自己的收敛判据为止。
cloudra/tools/core/xtb_path_runner.py:85    DEFAULT_REFINEMENT_RUNS = 100
```

```
cloudra/tools/core/xtb_path_runner.py:86    #: ``$path`` 输入文件名（与命令行一起落盘备查）
cloudra/tools/core/xtb_path_runner.py:87    PATH_INPUT_NAME = "path.inp"
cloudra/tools/core/xtb_path_runner.py:88
cloudra/tools/core/xtb_path_runner.py:89    #: 初猜来源标识（SOP §2：初猜是怎么来的必须写明，不许静默降级）。
cloudra/tools/core/xtb_path_runner.py:90    PATH_KIND_XTB = "xtb_path"
cloudra/tools/core/xtb_path_runner.py:91    PATH_KIND_IDPP = "idpp_fallback"
cloudra/tools/core/xtb_path_runner.py:92
cloudra/tools/core/xtb_path_runner.py:93    PathTermination = Literal[
cloudra/tools/core/xtb_path_runner.py:94        "completed",
cloudra/tools/core/xtb_path_runner.py:95        "executable_missing",
cloudra/tools/core/xtb_path_runner.py:96        "timeout",
cloudra/tools/core/xtb_path_runner.py:97        "invalid_endpoints",
cloudra/tools/core/xtb_path_runner.py:98        "incomplete_artifacts",
cloudra/tools/core/xtb_path_runner.py:99        "path_not_converged",
cloudra/tools/core/xtb_path_runner.py:100       "failed",
cloudra/tools/core/xtb_path_runner.py:101       "unknown_failure",
cloudra/tools/core/xtb_path_runner.py:102   ]
cloudra/tools/core/xtb_path_runner.py:103
cloudra/tools/core/xtb_path_runner.py:104   #: 引擎**确实启动过**的那些终止类别（子进程起来了、stdout 落盘、可能已有采样帧）：
cloudra/tools/core/xtb_path_runner.py:105   #: "跑了但没产出可用初猜"与"根本没搜过"必须分开上报——GH #39 实测
cloudra/tools/core/xtb_path_runner.py:106   #: ``incomplete_artifacts`` 时目录里已留下 ``xtbpath_1..24.xyz`` 可用采样，
cloudra/tools/core/xtb_path_runner.py:107   #: 报成 ``not_attempted`` 会让上层误判"根本没搜"。``executable_missing`` 与
cloudra/tools/core/xtb_path_runner.py:108   #: ``invalid_endpoints`` 都发生在子进程启动之前，不在本集合。
cloudra/tools/core/xtb_path_runner.py:109   XTB_ENGINE_STARTED_TERMINATIONS: frozenset[str] = frozenset({
cloudra/tools/core/xtb_path_runner.py:110       "timeout",
cloudra/tools/core/xtb_path_runner.py:111       "failed",
cloudra/tools/core/xtb_path_runner.py:112       "unknown_failure",
cloudra/tools/core/xtb_path_runner.py:113       "incomplete_artifacts",
cloudra/tools/core/xtb_path_runner.py:114       "path_not_converged",
cloudra/tools/core/xtb_path_runner.py:115   })
cloudra/tools/core/xtb_path_runner.py:116
cloudra/tools/core/xtb_path_runner.py:117   #: 引擎自己报的路径精修结局。``""`` = 日志里没有这两句终止语（本模块按未知处理，不默认成
    功）。
cloudra/tools/core/xtb_path_runner.py:118   RefinementTermination = Literal["", "converged", "cycles_exhausted"]
cloudra/tools/core/xtb_path_runner.py:119
cloudra/tools/core/xtb_path_runner.py:120
cloudra/tools/core/xtb_path_runner.py:121   class XtbPathRequest(BaseModel):
cloudra/tools/core/xtb_path_runner.py:122       """一次 MTD 路径搜索的请求。"""
cloudra/tools/core/xtb_path_runner.py:123
cloudra/tools/core/xtb_path_runner.py:124       reactant_xyz: str = Field(description="反应物几何（单帧 xyz 文本）")
cloudra/tools/core/xtb_path_runner.py:125       product_xyz: str = Field(description="产物几何（单帧 xyz 文本）；元素序列必须与反
    应物一致")
cloudra/tools/core/xtb_path_runner.py:126       output_dir: str = Field(description="持久运行目录的父目录；每次运行在其
    xtb_path_runs/ 下新建")
cloudra/tools/core/xtb_path_runner.py:127       charge: int = Field(default=0, description="总电荷（--chrg）")
cloudra/tools/core/xtb_path_runner.py:128       multiplicity: int = Field(default=1, description="自旋多重度 2S+1，透传为 --
    uhf = multiplicity-1")
cloudra/tools/core/xtb_path_runner.py:129       timeout: int = Field(default=900, description="子进程耗时上限（秒）；超时只终止本
    次运行自己的进程组")
cloudra/tools/core/xtb_path_runner.py:130       refinement_runs: int = Field(
cloudra/tools/core/xtb_path_runner.py:131           default=DEFAULT_REFINEMENT_RUNS, ge=1,
cloudra/tools/core/xtb_path_runner.py:132           description="路径精修的轮数上限（$path nrun）。**只是机时上限**：收敛由引擎自己
    判，"
cloudra/tools/core/xtb_path_runner.py:133                       "触到这个上限即报 path_not_converged。实测内建默认 3 轮远不够收敛，
    "
cloudra/tools/core/xtb_path_runner.py:134                       "不要把它调小",
cloudra/tools/core/xtb_path_runner.py:135       )
cloudra/tools/core/xtb_path_runner.py:136       run_label: str = Field(default="xtbpath", description="运行目录前缀")
cloudra/tools/core/xtb_path_runner.py:137       extra_args: list[str] = Field(default_factory=list, description="追加到命令行
    的原生选项")
```

```
cloudra/tools/core/xtb_path_runner.py:138
cloudra/tools/core/xtb_path_runner.py:139
cloudra/tools/core/xtb_path_runner.py:140   class XtbArtifact(BaseModel):
cloudra/tools/core/xtb_path_runner.py:141       name: str
cloudra/tools/core/xtb_path_runner.py:142       path: str
cloudra/tools/core/xtb_path_runner.py:143       size_bytes: int
cloudra/tools/core/xtb_path_runner.py:144
cloudra/tools/core/xtb_path_runner.py:145
cloudra/tools/core/xtb_path_runner.py:146   class PathFrame(BaseModel):
cloudra/tools/core/xtb_path_runner.py:147       """路径上的一帧。"""
cloudra/tools/core/xtb_path_runner.py:148
cloudra/tools/core/xtb_path_runner.py:149       frame_index: int
cloudra/tools/core/xtb_path_runner.py:150       rel_energy_kcal: Optional[float] = Field(
cloudra/tools/core/xtb_path_runner.py:151           default=None,
cloudra/tools/core/xtb_path_runner.py:152           description="文件注释里的能量，单位**已核实为 kcal/mol**，相对输入几何；取不到时为 None",
cloudra/tools/core/xtb_path_runner.py:153       )
cloudra/tools/core/xtb_path_runner.py:154       positions: list[list[float]]
cloudra/tools/core/xtb_path_runner.py:155
cloudra/tools/core/xtb_path_runner.py:156
cloudra/tools/core/xtb_path_runner.py:157   class XtbPathResult(BaseModel):
cloudra/tools/core/xtb_path_runner.py:158       """路径搜索结果。进程退出与"能不能用来当初猜"分开表达。"""
cloudra/tools/core/xtb_path_runner.py:159
cloudra/tools/core/xtb_path_runner.py:160       run_dir: str
cloudra/tools/core/xtb_path_runner.py:161       command: list[str] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:162       returncode: Optional[int] = None
cloudra/tools/core/xtb_path_runner.py:163       path_kind: Literal["xtb_path"] = PATH_KIND_XTB
cloudra/tools/core/xtb_path_runner.py:164       termination_reason: PathTermination = "unknown_failure"
cloudra/tools/core/xtb_path_runner.py:165       refinement_runs: int = Field(default=0, description="本次传给引擎的路径精修轮数上限")
cloudra/tools/core/xtb_path_runner.py:166       refinement_termination: RefinementTermination = Field(
cloudra/tools/core/xtb_path_runner.py:167           default="",
cloudra/tools/core/xtb_path_runner.py:168           description="引擎自己报的精修结局：converged（引擎判收敛）/ cycles_exhausted（触顶截断）",
cloudra/tools/core/xtb_path_runner.py:169       )
cloudra/tools/core/xtb_path_runner.py:170       usable_as_guess: bool = Field(
cloudra/tools/core/xtb_path_runner.py:171           default=False,
cloudra/tools/core/xtb_path_runner.py:172           description="是否产出了可用的初猜（路径 + TS 候选齐全且几何自洽）。"
cloudra/tools/core/xtb_path_runner.py:173                       "**这不等于已验证**：初猜仍需频率/IRC 判定",
cloudra/tools/core/xtb_path_runner.py:174       )
cloudra/tools/core/xtb_path_runner.py:175       symbols: list[str] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:176       frames: list[PathFrame] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:177       ts_candidate_energy_eh: Optional[float] = Field(
cloudra/tools/core/xtb_path_runner.py:178           default=None, description="TS 候选的**绝对**能量（Eh）；与路径帧的相对 kcal/mol 不是一个字段"
cloudra/tools/core/xtb_path_runner.py:179       )
cloudra/tools/core/xtb_path_runner.py:180       ts_candidate_positions: list[list[float]] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:181       reactant_path: str = ""
cloudra/tools/core/xtb_path_runner.py:182       product_path: str = ""
cloudra/tools/core/xtb_path_runner.py:183       command_path: str = ""
cloudra/tools/core/xtb_path_runner.py:184       path_input_path: str = ""
cloudra/tools/core/xtb_path_runner.py:185       versions_path: str = ""
cloudra/tools/core/xtb_path_runner.py:186       stdout_path: str = ""
cloudra/tools/core/xtb_path_runner.py:187       stderr_path: str = ""
cloudra/tools/core/xtb_path_runner.py:188       path_xyz_path: str = ""
cloudra/tools/core/xtb_path_runner.py:189       ts_xyz_path: str = ""
```

```
    artifacts: list[XtbArtifact] = Field(default_factory=list)
    partial_frames: list[str] = Field(
        default_factory=list,
        description="引擎异常退出时留下的逐迭代中间帧（xtbpath_<n>.xyz）。**不是初猜**___"
                    "只做可发现性：让上层知道有一批采样可自行检视（GH #39）",
    )
    evidence: list[str] = Field(default_factory=list, description="命中的逐字标记行（原样保留）")

    diagnostics: list[str] = Field(default_factory=list)
    message: str = ""


def build_path_input(refinement_runs: int) -> str:
    """``$path`` 输入块：把精修轮数上限显式交给引擎，**不使用内建默认 3 轮**。

    本机 6.7.1 实测：``$path`` 块被读取；它认得的键是 ``nrun`` / ``npoint`` / ``kpush`` /
    ``kpull``，其余（``npoints`` / ``alpha`` / ``step`` / ``time`` / ``seed`` 等）会被逐字拒掉
    （``the key '...' is not recognized by path``），因此这里只写实测可用的键。
    """
    if refinement_runs < 1:
        raise ValueError(f"refinement_runs must be >= 1, got {refinement_runs}")

    return f"$path\n   nrun={refinement_runs}\n$end\n"


def build_xtb_path_command(request: XtbPathRequest) -> list[str]:
    """构造命令行。电荷与自旋显式透传，绝不依赖引擎默认推断。"""
    if request.multiplicity < 1:
        raise ValueError(f"multiplicity must be >= 1 (2S+1), got {request.multiplicity}")

    return [
        "xtb", "reactant.xyz", "--path", "product.xyz", "--input",
        PATH_INPUT_NAME,
        "--chrg", str(request.charge), "--uhf", str(request.multiplicity - 1),
        *request.extra_args,
    ]


def list_partial_frames(run_dir: str) -> list[str]:
    """列出引擎留下的逐迭代中间帧（``xtbpath_<n>.xyz``），按序号升序。

    显式 token 扫描（前缀 + 纯数字 + 后缀），不用正则——``xtbpath_ts.xyz``（TS 候选）与
    ``xtbpath.xyz``（最终路径）因此天然被排除。
    """
    if not os.path.isdir(run_dir):
        return []
    found: list[tuple[int, str]] = []
    for name in os.listdir(run_dir):
        if not (name.startswith(PARTIAL_FRAME_PREFIX) and name.endswith(PARTIAL_FRAME_SUFFIX)):
            continue
        middle = name[len(PARTIAL_FRAME_PREFIX):-len(PARTIAL_FRAME_SUFFIX)]
        if not middle.isdigit():
            continue
        found.append((int(middle), os.path.join(run_dir, name)))
    found.sort()
    return [path for _, path in found]
```

```
cloudra/tools/core/xtb_path_runner.py:242
cloudra/tools/core/xtb_path_runner.py:243
cloudra/tools/core/xtb_path_runner.py:244   def _record_partial_frames(result: XtbPathResult) -> None:
cloudra/tools/core/xtb_path_runner.py:245       """把中间帧写进结果与诊断。**只做可发现性**：不作初猜（GH #39）。
cloudra/tools/core/xtb_path_runner.py:246
cloudra/tools/core/xtb_path_runner.py:247       崩溃退出的中间帧比"迭代到上限的截断路径"更可疑（引擎是异常退出的），因此不能放宽
cloudra/tools/core/xtb_path_runner.py:248       ``usable_as_guess``；但它们必须能被上层看见——否则连"这里有一批采样"都不知道。
cloudra/tools/core/xtb_path_runner.py:249       """
cloudra/tools/core/xtb_path_runner.py:250       result.partial_frames = list_partial_frames(result.run_dir)
cloudra/tools/core/xtb_path_runner.py:251       if result.partial_frames:
cloudra/tools/core/xtb_path_runner.py:252           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:253               f"{len(result.partial_frames)} partial iteration frame(s) were left
     behind "
cloudra/tools/core/xtb_path_runner.py:254               f"(first: {os.path.basename(result.partial_frames[0])}); they are
    sampling "
cloudra/tools/core/xtb_path_runner.py:255               f"artifacts, not a usable TS guess"
cloudra/tools/core/xtb_path_runner.py:256           )
cloudra/tools/core/xtb_path_runner.py:257
cloudra/tools/core/xtb_path_runner.py:258
cloudra/tools/core/xtb_path_runner.py:259   def _symbols_and_positions(xyz_text: str, *, label: str) -> tuple[list[str],
    list[list[float]]]:
cloudra/tools/core/xtb_path_runner.py:260       """解析单帧 xyz；不完整即 fail-loud。"""
cloudra/tools/core/xtb_path_runner.py:261       lines = xyz_text.splitlines()
cloudra/tools/core/xtb_path_runner.py:262       if not lines:
cloudra/tools/core/xtb_path_runner.py:263           raise ValueError(f"{label}: empty xyz text")
cloudra/tools/core/xtb_path_runner.py:264       try:
cloudra/tools/core/xtb_path_runner.py:265           count = int(lines[0].strip())
cloudra/tools/core/xtb_path_runner.py:266       except ValueError as exc:
cloudra/tools/core/xtb_path_runner.py:267           raise ValueError(f"{label}: first line must be the atom count, got {
    lines[0]!r}") from exc
cloudra/tools/core/xtb_path_runner.py:268       if count < 1:
cloudra/tools/core/xtb_path_runner.py:269           raise ValueError(f"{label}: declares {count} atoms")
cloudra/tools/core/xtb_path_runner.py:270       if len(lines) < 2 + count:
cloudra/tools/core/xtb_path_runner.py:271           raise ValueError(f"{label}: declares {count} atoms but has too few
    lines")
cloudra/tools/core/xtb_path_runner.py:272       symbols: list[str] = []
cloudra/tools/core/xtb_path_runner.py:273       positions: list[list[float]] = []
cloudra/tools/core/xtb_path_runner.py:274       for line in lines[2: 2 + count]:
cloudra/tools/core/xtb_path_runner.py:275           parts = line.split()
cloudra/tools/core/xtb_path_runner.py:276           if len(parts) < 4:
cloudra/tools/core/xtb_path_runner.py:277               raise ValueError(f"{label}: malformed coordinate line {line!r}")
cloudra/tools/core/xtb_path_runner.py:278           symbols.append(parts[0])
cloudra/tools/core/xtb_path_runner.py:279           positions.append([float(parts[1]), float(parts[2]), float(parts[3])])
cloudra/tools/core/xtb_path_runner.py:280       return symbols, positions
cloudra/tools/core/xtb_path_runner.py:281
cloudra/tools/core/xtb_path_runner.py:282
cloudra/tools/core/xtb_path_runner.py:283   def _parse_path_frames(path: str) -> list[PathFrame]:
cloudra/tools/core/xtb_path_runner.py:284       """解析 ``xtbpath.xyz``。注释里的能量是**相对 kcal/mol**，取不到就留 None，不猜。
    """
cloudra/tools/core/xtb_path_runner.py:285       with open(path, encoding="utf-8", errors="replace") as handle:
cloudra/tools/core/xtb_path_runner.py:286           lines = handle.read().splitlines()
cloudra/tools/core/xtb_path_runner.py:287       frames: list[PathFrame] = []
cloudra/tools/core/xtb_path_runner.py:288       index = 0
cloudra/tools/core/xtb_path_runner.py:289       while index < len(lines):
cloudra/tools/core/xtb_path_runner.py:290           if not lines[index].strip():
cloudra/tools/core/xtb_path_runner.py:291               index += 1
cloudra/tools/core/xtb_path_runner.py:292               continue
cloudra/tools/core/xtb_path_runner.py:293           try:
```

```
cloudra/tools/core/xtb_path_runner.py:294             count = int(lines[index].strip())
cloudra/tools/core/xtb_path_runner.py:295         except ValueError:
cloudra/tools/core/xtb_path_runner.py:296             break
cloudra/tools/core/xtb_path_runner.py:297         comment = lines[index + 1] if index + 1 < len(lines) else ""
cloudra/tools/core/xtb_path_runner.py:298         rel: Optional[float] = None
cloudra/tools/core/xtb_path_runner.py:299         tokens = comment.split()
cloudra/tools/core/xtb_path_runner.py:300         # 形如 ``energy: <float> xtb: <version>``；逐 token 取，不做模糊匹配
cloudra/tools/core/xtb_path_runner.py:301         if len(tokens) >= 2 and tokens[0] == "energy:":
cloudra/tools/core/xtb_path_runner.py:302             try:
cloudra/tools/core/xtb_path_runner.py:303                 rel = float(tokens[1])
cloudra/tools/core/xtb_path_runner.py:304             except ValueError:
cloudra/tools/core/xtb_path_runner.py:305                 rel = None
cloudra/tools/core/xtb_path_runner.py:306         block = lines[index + 2: index + 2 + count]
cloudra/tools/core/xtb_path_runner.py:307         if len(block) != count:
cloudra/tools/core/xtb_path_runner.py:308             break
cloudra/tools/core/xtb_path_runner.py:309         positions = []
cloudra/tools/core/xtb_path_runner.py:310         for line in block:
cloudra/tools/core/xtb_path_runner.py:311             parts = line.split()
cloudra/tools/core/xtb_path_runner.py:312             if len(parts) < 4:
cloudra/tools/core/xtb_path_runner.py:313                 raise ValueError(f"{path}: malformed coordinate line {line!r}")
cloudra/tools/core/xtb_path_runner.py:314             positions.append([float(parts[1]), float(parts[2]), float(parts[3])
])
cloudra/tools/core/xtb_path_runner.py:315         frames.append(PathFrame(frame_index=len(frames), rel_energy_kcal=rel,
positions=positions))
cloudra/tools/core/xtb_path_runner.py:316         index += 2 + count
cloudra/tools/core/xtb_path_runner.py:317     if not frames:
cloudra/tools/core/xtb_path_runner.py:318         raise ValueError(f"{path}: no complete frame could be read")
cloudra/tools/core/xtb_path_runner.py:319     return frames
cloudra/tools/core/xtb_path_runner.py:320
cloudra/tools/core/xtb_path_runner.py:321
cloudra/tools/core/xtb_path_runner.py:322 def _parse_ts_candidate(path: str) -> tuple[float, list[list[float]]]:
cloudra/tools/core/xtb_path_runner.py:323     """解析 ``xtbpath_ts.xyz``。注释里的能量是**绝对 Eh**——与路径帧的单位不同。"""
cloudra/tools/core/xtb_path_runner.py:324     with open(path, encoding="utf-8", errors="replace") as handle:
cloudra/tools/core/xtb_path_runner.py:325         lines = handle.read().splitlines()
cloudra/tools/core/xtb_path_runner.py:326     if len(lines) < 3:
cloudra/tools/core/xtb_path_runner.py:327         raise ValueError(f"{path}: too few lines for a single frame")
cloudra/tools/core/xtb_path_runner.py:328     count = int(lines[0].strip())
cloudra/tools/core/xtb_path_runner.py:329     tokens = lines[1].split()
cloudra/tools/core/xtb_path_runner.py:330     if len(tokens) < 2 or tokens[0] != "energy:":
cloudra/tools/core/xtb_path_runner.py:331         raise ValueError(
cloudra/tools/core/xtb_path_runner.py:332             f"{path}: comment line must start with 'energy: <Eh>', got {lines
[1]!r}"
cloudra/tools/core/xtb_path_runner.py:333         )
cloudra/tools/core/xtb_path_runner.py:334     energy = float(tokens[1])
cloudra/tools/core/xtb_path_runner.py:335     block = lines[2: 2 + count]
cloudra/tools/core/xtb_path_runner.py:336     if len(block) != count:
cloudra/tools/core/xtb_path_runner.py:337         raise ValueError(f"{path}: declares {count} atoms but only {len(block)}
 coordinates follow")
cloudra/tools/core/xtb_path_runner.py:338     positions = []
cloudra/tools/core/xtb_path_runner.py:339     for line in block:
cloudra/tools/core/xtb_path_runner.py:340         parts = line.split()
cloudra/tools/core/xtb_path_runner.py:341         if len(parts) < 4:
cloudra/tools/core/xtb_path_runner.py:342             raise ValueError(f"{path}: malformed coordinate line {line!r}")
cloudra/tools/core/xtb_path_runner.py:343         positions.append([float(parts[1]), float(parts[2]), float(parts[3])])
cloudra/tools/core/xtb_path_runner.py:344     return energy, positions
cloudra/tools/core/xtb_path_runner.py:345
```

```
cloudra/tools/core/xtb_path_runner.py:346
cloudra/tools/core/xtb_path_runner.py:347   class PathMarkers(BaseModel):
cloudra/tools/core/xtb_path_runner.py:348       """日志里命中的逐字标记。"""
cloudra/tools/core/xtb_path_runner.py:349
cloudra/tools/core/xtb_path_runner.py:350       evidence: list[str] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:351       normal_termination: bool = False
cloudra/tools/core/xtb_path_runner.py:352       path_finder: bool = False
cloudra/tools/core/xtb_path_runner.py:353       refinement_termination: RefinementTermination = ""
cloudra/tools/core/xtb_path_runner.py:354
cloudra/tools/core/xtb_path_runner.py:355
cloudra/tools/core/xtb_path_runner.py:356   def _scan_markers(lines: list[str]) -> PathMarkers:
cloudra/tools/core/xtb_path_runner.py:357       """精确 token 扫描。两条终止语**分开**判定：收敛与触顶绝不等价。
cloudra/tools/core/xtb_path_runner.py:358
cloudra/tools/core/xtb_path_runner.py:359       barrier 行每轮精修都打印一次，只保留**最后一次**（更靠后的那个才是当前精修程度）。
cloudra/tools/core/xtb_path_runner.py:360       """
cloudra/tools/core/xtb_path_runner.py:361       markers = PathMarkers()
cloudra/tools/core/xtb_path_runner.py:362       last_forward = ""
cloudra/tools/core/xtb_path_runner.py:363       last_backward = ""
cloudra/tools/core/xtb_path_runner.py:364       for line in lines:
cloudra/tools/core/xtb_path_runner.py:365           stripped = line.strip()
cloudra/tools/core/xtb_path_runner.py:366           if MARKER_NORMAL_TERMINATION in stripped:
cloudra/tools/core/xtb_path_runner.py:367               markers.evidence.append(stripped)
cloudra/tools/core/xtb_path_runner.py:368               markers.normal_termination = True
cloudra/tools/core/xtb_path_runner.py:369           elif stripped.startswith(MARKER_FINISHED_RUN.strip()):
cloudra/tools/core/xtb_path_runner.py:370               markers.evidence.append(stripped)
cloudra/tools/core/xtb_path_runner.py:371           elif MARKER_PATH_FINDER.strip() in stripped:
cloudra/tools/core/xtb_path_runner.py:372               markers.evidence.append(stripped)
cloudra/tools/core/xtb_path_runner.py:373               markers.path_finder = True
cloudra/tools/core/xtb_path_runner.py:374           elif MARKER_PATH_CONVERGED in stripped:
cloudra/tools/core/xtb_path_runner.py:375               markers.evidence.append(stripped)
cloudra/tools/core/xtb_path_runner.py:376               markers.refinement_termination = "converged"
cloudra/tools/core/xtb_path_runner.py:377           elif MARKER_PATH_CYCLES_EXHAUSTED in stripped:
cloudra/tools/core/xtb_path_runner.py:378               markers.evidence.append(stripped)
cloudra/tools/core/xtb_path_runner.py:379               markers.refinement_termination = "cycles_exhausted"
cloudra/tools/core/xtb_path_runner.py:380           elif MARKER_FORWARD_BARRIER in stripped:
cloudra/tools/core/xtb_path_runner.py:381               last_forward = stripped
cloudra/tools/core/xtb_path_runner.py:382           elif MARKER_BACKWARD_BARRIER in stripped:
cloudra/tools/core/xtb_path_runner.py:383               last_backward = stripped
cloudra/tools/core/xtb_path_runner.py:384       for extra in (last_forward, last_backward):
cloudra/tools/core/xtb_path_runner.py:385           if extra:
cloudra/tools/core/xtb_path_runner.py:386               markers.evidence.append(extra)
cloudra/tools/core/xtb_path_runner.py:387       markers.evidence = markers.evidence[:MAX_EVIDENCE_LINES]
cloudra/tools/core/xtb_path_runner.py:388       return markers
cloudra/tools/core/xtb_path_runner.py:389
cloudra/tools/core/xtb_path_runner.py:390
cloudra/tools/core/xtb_path_runner.py:391   def run_xtb_path(request: XtbPathRequest) -> XtbPathResult:
cloudra/tools/core/xtb_path_runner.py:392       """在持久运行目录里跑一次 MTD 路径搜索。
cloudra/tools/core/xtb_path_runner.py:393
cloudra/tools/core/xtb_path_runner.py:394       同步函数（子进程 + 文件 IO）；异步调用方必须经 ``asyncio.to_thread`` 卸载。
cloudra/tools/core/xtb_path_runner.py:395       """
cloudra/tools/core/xtb_path_runner.py:396       try:
cloudra/tools/core/xtb_path_runner.py:397           reactant_symbols, _ = _symbols_and_positions(request.reactant_xyz,
    label="reactant")
```

```
cloudra/tools/core/xtb_path_runner.py:398           product_symbols, _ = _symbols_and_positions(request.product_xyz, label="product")
cloudra/tools/core/xtb_path_runner.py:399       except ValueError as exc:
cloudra/tools/core/xtb_path_runner.py:400           # 端点先自检：几何不完整或元素序列不一致时，跑引擎只会浪费机时
cloudra/tools/core/xtb_path_runner.py:401           output_dir = request.output_dir
cloudra/tools/core/xtb_path_runner.py:402           os.makedirs(output_dir, exist_ok=True)
cloudra/tools/core/xtb_path_runner.py:403           run_dir = next_run_dir(output_dir, RUNS_DIR_NAME, request.run_label)
cloudra/tools/core/xtb_path_runner.py:404           return XtbPathResult(
cloudra/tools/core/xtb_path_runner.py:405               run_dir=run_dir, termination_reason="invalid_endpoints",
cloudra/tools/core/xtb_path_runner.py:406               message=f"the endpoints were rejected before starting xtb: {exc}",
cloudra/tools/core/xtb_path_runner.py:407               diagnostics=[str(exc)],
cloudra/tools/core/xtb_path_runner.py:408           )
cloudra/tools/core/xtb_path_runner.py:409
cloudra/tools/core/xtb_path_runner.py:410       command = build_xtb_path_command(request)
cloudra/tools/core/xtb_path_runner.py:411       run_dir = next_run_dir(request.output_dir, RUNS_DIR_NAME, request.run_label)
cloudra/tools/core/xtb_path_runner.py:412       reactant_path = os.path.join(run_dir, "reactant.xyz")
cloudra/tools/core/xtb_path_runner.py:413       product_path = os.path.join(run_dir, "product.xyz")
cloudra/tools/core/xtb_path_runner.py:414       command_path = os.path.join(run_dir, "command.txt")
cloudra/tools/core/xtb_path_runner.py:415       path_input_path = os.path.join(run_dir, PATH_INPUT_NAME)
cloudra/tools/core/xtb_path_runner.py:416       versions_path = os.path.join(run_dir, "versions.txt")
cloudra/tools/core/xtb_path_runner.py:417       stdout_path = os.path.join(run_dir, "stdout.log")
cloudra/tools/core/xtb_path_runner.py:418       stderr_path = os.path.join(run_dir, "stderr.log")
cloudra/tools/core/xtb_path_runner.py:419
cloudra/tools/core/xtb_path_runner.py:420       atomic_write_text(reactant_path, request.reactant_xyz)
cloudra/tools/core/xtb_path_runner.py:421       atomic_write_text(product_path, request.product_xyz)
cloudra/tools/core/xtb_path_runner.py:422       atomic_write_text(command_path, " ".join(command) + "\n")
cloudra/tools/core/xtb_path_runner.py:423       atomic_write_text(path_input_path, build_path_input(request.refinement_runs))
cloudra/tools/core/xtb_path_runner.py:424       atomic_write_text(versions_path, capture_versions(("xtb",), ("xtb version",)))
cloudra/tools/core/xtb_path_runner.py:425
cloudra/tools/core/xtb_path_runner.py:426       result = XtbPathResult(
cloudra/tools/core/xtb_path_runner.py:427           run_dir=run_dir, command=command, symbols=reactant_symbols,
cloudra/tools/core/xtb_path_runner.py:428           reactant_path=reactant_path, product_path=product_path,
cloudra/tools/core/xtb_path_runner.py:429           command_path=command_path, path_input_path=path_input_path,
versions_path=versions_path,
cloudra/tools/core/xtb_path_runner.py:430           refinement_runs=request.refinement_runs,
cloudra/tools/core/xtb_path_runner.py:431           stdout_path=stdout_path, stderr_path=stderr_path,
cloudra/tools/core/xtb_path_runner.py:432           path_xyz_path=os.path.join(run_dir, PATH_XYZ_NAME),
cloudra/tools/core/xtb_path_runner.py:433           ts_xyz_path=os.path.join(run_dir, TS_XYZ_NAME),
cloudra/tools/core/xtb_path_runner.py:434       )
cloudra/tools/core/xtb_path_runner.py:435
cloudra/tools/core/xtb_path_runner.py:436       if reactant_symbols != product_symbols:
cloudra/tools/core/xtb_path_runner.py:437           result.termination_reason = "invalid_endpoints"
cloudra/tools/core/xtb_path_runner.py:438           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:439               f"reactant and product element sequences differ ({reactant_symbols} vs "
cloudra/tools/core/xtb_path_runner.py:440               f"{product_symbols}); the path search needs one consistent atom set"
cloudra/tools/core/xtb_path_runner.py:441           )
cloudra/tools/core/xtb_path_runner.py:442           result.message = "Endpoints rejected: the element sequences differ."
cloudra/tools/core/xtb_path_runner.py:443           result.artifacts = [
cloudra/tools/core/xtb_path_runner.py:444               XtbArtifact(name=name, path=path, size_bytes=size)
cloudra/tools/core/xtb_path_runner.py:445               for name, path, size in list_artifact_files(run_dir)
cloudra/tools/core/xtb_path_runner.py:446           ]
cloudra/tools/core/xtb_path_runner.py:447           return result
cloudra/tools/core/xtb_path_runner.py:448
cloudra/tools/core/xtb_path_runner.py:449       import shutil
```

```
cloudra/tools/core/xtb_path_runner.py:450
cloudra/tools/core/xtb_path_runner.py:451       if not shutil.which("xtb"):
cloudra/tools/core/xtb_path_runner.py:452           result.termination_reason = "executable_missing"
cloudra/tools/core/xtb_path_runner.py:453           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:454               "the 'xtb' executable was not found on PATH; no run was started.
The caller should "
cloudra/tools/core/xtb_path_runner.py:455               f"fall back to IDPP interpolation and report path_kind={
PATH_KIND_IDPP!r}"
cloudra/tools/core/xtb_path_runner.py:456           )
cloudra/tools/core/xtb_path_runner.py:457           atomic_write_text(stdout_path, "")
cloudra/tools/core/xtb_path_runner.py:458           atomic_write_text(stderr_path, "")
cloudra/tools/core/xtb_path_runner.py:459           result.artifacts = [
cloudra/tools/core/xtb_path_runner.py:460               XtbArtifact(name=name, path=path, size_bytes=size)
cloudra/tools/core/xtb_path_runner.py:461               for name, path, size in list_artifact_files(run_dir)
cloudra/tools/core/xtb_path_runner.py:462           ]
cloudra/tools/core/xtb_path_runner.py:463           result.message = "xtb executable not found; no path search was started
."
cloudra/tools/core/xtb_path_runner.py:464           return result
cloudra/tools/core/xtb_path_runner.py:465
cloudra/tools/core/xtb_path_runner.py:466       timed_out = False
cloudra/tools/core/xtb_path_runner.py:467       proc = subprocess.Popen(
cloudra/tools/core/xtb_path_runner.py:468           command, cwd=run_dir, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
cloudra/tools/core/xtb_path_runner.py:469           text=True, start_new_session=True,
cloudra/tools/core/xtb_path_runner.py:470       )
cloudra/tools/core/xtb_path_runner.py:471       try:
cloudra/tools/core/xtb_path_runner.py:472           stdout, stderr = proc.communicate(timeout=request.timeout)
cloudra/tools/core/xtb_path_runner.py:473           returncode = proc.returncode
cloudra/tools/core/xtb_path_runner.py:474       except subprocess.TimeoutExpired:
cloudra/tools/core/xtb_path_runner.py:475           timed_out = True
cloudra/tools/core/xtb_path_runner.py:476           terminate_process_group(proc)
cloudra/tools/core/xtb_path_runner.py:477           stdout, stderr = proc.communicate()
cloudra/tools/core/xtb_path_runner.py:478           returncode = proc.returncode
cloudra/tools/core/xtb_path_runner.py:479           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:480               f"the xtb path search exceeded the {request.timeout}s timeout; this
 run's own process "
cloudra/tools/core/xtb_path_runner.py:481               "group was terminated (its logs and intermediate files are kept)"
cloudra/tools/core/xtb_path_runner.py:482           )
cloudra/tools/core/xtb_path_runner.py:483
cloudra/tools/core/xtb_path_runner.py:484       atomic_write_text(stdout_path, stdout or "")
cloudra/tools/core/xtb_path_runner.py:485       atomic_write_text(stderr_path, stderr or "")
cloudra/tools/core/xtb_path_runner.py:486       result.returncode = returncode
cloudra/tools/core/xtb_path_runner.py:487       result.artifacts = [
cloudra/tools/core/xtb_path_runner.py:488           XtbArtifact(name=name, path=path, size_bytes=size)
cloudra/tools/core/xtb_path_runner.py:489           for name, path, size in list_artifact_files(run_dir)
cloudra/tools/core/xtb_path_runner.py:490       ]
cloudra/tools/core/xtb_path_runner.py:491
cloudra/tools/core/xtb_path_runner.py:492       # 实测（本机 xtb 6.7.1）：``normal termination of xtb`` 落在 **stderr**、而计时
块落在
cloudra/tools/core/xtb_path_runner.py:493       # stdout。因此标记必须在两个流上一起扫——只看 stdout 会把正常的运行判成失败。
cloudra/tools/core/xtb_path_runner.py:494       markers = _scan_markers(
cloudra/tools/core/xtb_path_runner.py:495           (stdout or "").splitlines() + (stderr or "").splitlines()
cloudra/tools/core/xtb_path_runner.py:496       )
cloudra/tools/core/xtb_path_runner.py:497       result.evidence = markers.evidence
cloudra/tools/core/xtb_path_runner.py:498       result.refinement_termination = markers.refinement_termination
cloudra/tools/core/xtb_path_runner.py:499
cloudra/tools/core/xtb_path_runner.py:500       if timed_out:
cloudra/tools/core/xtb_path_runner.py:501           result.termination_reason = "timeout"
```

```
cloudra/tools/core/xtb_path_runner.py:502           result.message = (
cloudra/tools/core/xtb_path_runner.py:503               f"the xtb path search timed out after {request.timeout}s (
returncode={returncode}). "
cloudra/tools/core/xtb_path_runner.py:504               f"Full log: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:505           )
cloudra/tools/core/xtb_path_runner.py:506           return result
cloudra/tools/core/xtb_path_runner.py:507
cloudra/tools/core/xtb_path_runner.py:508       if returncode != 0:
cloudra/tools/core/xtb_path_runner.py:509           result.termination_reason = "failed"
cloudra/tools/core/xtb_path_runner.py:510           tail = "\n".join((stdout or "").splitlines()[-15:]) or (stderr or "")
[-2000:]
cloudra/tools/core/xtb_path_runner.py:511           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:512               "xtb exited non-zero; raw log tail follows (no cause is guessed)"
cloudra/tools/core/xtb_path_runner.py:513           )
cloudra/tools/core/xtb_path_runner.py:514           result.diagnostics.append(tail)
cloudra/tools/core/xtb_path_runner.py:515           result.message = (
cloudra/tools/core/xtb_path_runner.py:516               f"the xtb path search failed (returncode={returncode}). Full log: {
stdout_path}"
cloudra/tools/core/xtb_path_runner.py:517           )
cloudra/tools/core/xtb_path_runner.py:518           return result
cloudra/tools/core/xtb_path_runner.py:519
cloudra/tools/core/xtb_path_runner.py:520       if not markers.normal_termination:
cloudra/tools/core/xtb_path_runner.py:521           result.termination_reason = "unknown_failure"
cloudra/tools/core/xtb_path_runner.py:522           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:523               f"xtb exited 0 but never printed its normal-termination marker "
cloudra/tools/core/xtb_path_runner.py:524               f"({MARKER_NORMAL_TERMINATION!r}, which this build writes to stderr
); refusing to treat "
cloudra/tools/core/xtb_path_runner.py:525               "the exit code alone as success"
cloudra/tools/core/xtb_path_runner.py:526           )
cloudra/tools/core/xtb_path_runner.py:527           result.message = f"xtb did not report normal termination. Full log: {
stdout_path}"
cloudra/tools/core/xtb_path_runner.py:528           return result
cloudra/tools/core/xtb_path_runner.py:529
cloudra/tools/core/xtb_path_runner.py:530       if not markers.path_finder:
cloudra/tools/core/xtb_path_runner.py:531           result.termination_reason = "unknown_failure"
cloudra/tools/core/xtb_path_runner.py:532           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:533               f"xtb exited 0 but never printed its path-finder timing block "
cloudra/tools/core/xtb_path_runner.py:534               f"({MARKER_PATH_FINDER.strip()!r}); refusing to treat the exit code
 alone as success"
cloudra/tools/core/xtb_path_runner.py:535           )
cloudra/tools/core/xtb_path_runner.py:536           result.message = f"xtb did not report a completed path search. Full log
: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:537           return result
cloudra/tools/core/xtb_path_runner.py:538
cloudra/tools/core/xtb_path_runner.py:539       # 产物完整性：路径与 TS 候选都要在，且几何自洽——"文件在"不构成成功判据
cloudra/tools/core/xtb_path_runner.py:540       try:
cloudra/tools/core/xtb_path_runner.py:541           frames = _parse_path_frames(result.path_xyz_path)
cloudra/tools/core/xtb_path_runner.py:542           ts_energy, ts_positions = _parse_ts_candidate(result.ts_xyz_path)
cloudra/tools/core/xtb_path_runner.py:543       except (OSError, ValueError) as exc:
cloudra/tools/core/xtb_path_runner.py:544           result.termination_reason = "incomplete_artifacts"
cloudra/tools/core/xtb_path_runner.py:545           result.diagnostics.append(f"path artifacts are unusable: {exc}")
cloudra/tools/core/xtb_path_runner.py:546           _record_partial_frames(result)
cloudra/tools/core/xtb_path_runner.py:547           result.message = (
cloudra/tools/core/xtb_path_runner.py:548               f"xtb reported normal termination but its path artifacts are
unusable ({exc}). "
cloudra/tools/core/xtb_path_runner.py:549               f"Full log: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:550           )
cloudra/tools/core/xtb_path_runner.py:551           return result
cloudra/tools/core/xtb_path_runner.py:552
cloudra/tools/core/xtb_path_runner.py:553       for frame in frames:
```

```
554 cloudra/tools/core/xtb_path_runner.py:554           if len(frame.positions) != len(reactant_symbols):
555 cloudra/tools/core/xtb_path_runner.py:555               result.termination_reason = "incomplete_artifacts"
556 cloudra/tools/core/xtb_path_runner.py:556               result.diagnostics.append(
557 cloudra/tools/core/xtb_path_runner.py:557                   f"frame {frame.frame_index} has {len(frame.positions)} atoms
    but the endpoints have "
558 cloudra/tools/core/xtb_path_runner.py:558                   f"{len(reactant_symbols)}"
559 cloudra/tools/core/xtb_path_runner.py:559               )
560 cloudra/tools/core/xtb_path_runner.py:560               result.message = "xtb reported normal termination but the path
    frames are inconsistent."
561 cloudra/tools/core/xtb_path_runner.py:561               return result
562 cloudra/tools/core/xtb_path_runner.py:562       if len(frames) < 2:
563 cloudra/tools/core/xtb_path_runner.py:563           result.termination_reason = "incomplete_artifacts"
564 cloudra/tools/core/xtb_path_runner.py:564           result.diagnostics.append(
565 cloudra/tools/core/xtb_path_runner.py:565               f"the path has only {len(frames)} frame(s); a path needs at least
    the two endpoints"
566 cloudra/tools/core/xtb_path_runner.py:566           )
567 cloudra/tools/core/xtb_path_runner.py:567           result.message = "xtb reported normal termination but produced no
    usable path."
568 cloudra/tools/core/xtb_path_runner.py:568           return result
569 cloudra/tools/core/xtb_path_runner.py:569       if len(ts_positions) != len(reactant_symbols):
570 cloudra/tools/core/xtb_path_runner.py:570           result.termination_reason = "incomplete_artifacts"
571 cloudra/tools/core/xtb_path_runner.py:571           result.diagnostics.append(
572 cloudra/tools/core/xtb_path_runner.py:572               f"the TS candidate has {len(ts_positions)} atoms but the endpoints
    have "
573 cloudra/tools/core/xtb_path_runner.py:573               f"{len(reactant_symbols)}"
574 cloudra/tools/core/xtb_path_runner.py:574           )
575 cloudra/tools/core/xtb_path_runner.py:575           result.message = "xtb reported normal termination but the TS candidate
    is inconsistent."
576 cloudra/tools/core/xtb_path_runner.py:576           return result
577 cloudra/tools/core/xtb_path_runner.py:577
578 cloudra/tools/core/xtb_path_runner.py:578       result.frames = frames
579 cloudra/tools/core/xtb_path_runner.py:579       result.ts_candidate_energy_eh = ts_energy
580 cloudra/tools/core/xtb_path_runner.py:580       result.ts_candidate_positions = ts_positions
581 cloudra/tools/core/xtb_path_runner.py:581
582 cloudra/tools/core/xtb_path_runner.py:582       if markers.refinement_termination == "cycles_exhausted":
583 cloudra/tools/core/xtb_path_runner.py:583           # **被截断的路径不是初猜**：实测这种路径的最高帧落在反应物自己的盆地里
584 cloudra/tools/core/xtb_path_runner.py:584           # （离起点只有 21°、相对能 44.865 kcal/mol，做 OptTS 跑 56 轮不收敛）。
585 cloudra/tools/core/xtb_path_runner.py:585           # 这里把截断的现场（路径帧 + TS 候选）保留下来供复查，但明确标成不可用。
586 cloudra/tools/core/xtb_path_runner.py:586           result.termination_reason = "path_not_converged"
587 cloudra/tools/core/xtb_path_runner.py:587           result.usable_as_guess = False
588 cloudra/tools/core/xtb_path_runner.py:588           result.diagnostics.append(
589 cloudra/tools/core/xtb_path_runner.py:589               f"the engine stopped its path refinement at the {request.
    refinement_runs}-run cap "
590 cloudra/tools/core/xtb_path_runner.py:590               f"({MARKER_PATH_CYCLES_EXHAUSTED!r}); that is a truncated path, not
     a converged one. "
591 cloudra/tools/core/xtb_path_runner.py:591               "The frames and TS candidate are kept for inspection only. Measured
     consequence of "
592 cloudra/tools/core/xtb_path_runner.py:592               "accepting a truncated path (docs/evidence/ts_guess_mtd_artifact/):
     its highest frame "
593 cloudra/tools/core/xtb_path_runner.py:593               "sits in the reactant's own basin, 21 deg from the start, and an
    ORCA OptTS started "
594 cloudra/tools/core/xtb_path_runner.py:594               "from it never converged (56 cycles, slid back to the minimum).
    Raise "
595 cloudra/tools/core/xtb_path_runner.py:595               "`refinement_runs` and rerun"
596 cloudra/tools/core/xtb_path_runner.py:596           )
597 cloudra/tools/core/xtb_path_runner.py:597           result.message = (
598 cloudra/tools/core/xtb_path_runner.py:598               f"the xTB path refinement hit its {request.refinement_runs}-run cap
     before the engine "
599 cloudra/tools/core/xtb_path_runner.py:599               f"declared convergence, so no usable TS guess was produced. Raise `
    refinement_runs` and "
600 cloudra/tools/core/xtb_path_runner.py:600               f"rerun. Run directory (truncated path kept for inspection): {
    run_dir}"
601 cloudra/tools/core/xtb_path_runner.py:601           )
602 cloudra/tools/core/xtb_path_runner.py:602           return result
603 cloudra/tools/core/xtb_path_runner.py:603
604 cloudra/tools/core/xtb_path_runner.py:604       if markers.refinement_termination != "converged":
605 cloudra/tools/core/xtb_path_runner.py:605           result.termination_reason = "unknown_failure"
```

```
cloudra/tools/core/xtb_path_runner.py:606           result.usable_as_guess = False
cloudra/tools/core/xtb_path_runner.py:607           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:608               "xtb exited 0 and produced path artifacts, but its log carries neither refinement "
cloudra/tools/core/xtb_path_runner.py:609               f"termination line ({MARKER_PATH_CONVERGED!r} / {MARKER_PATH_CYCLES_EXHAUSTED!r}); "
cloudra/tools/core/xtb_path_runner.py:610               "whether the path refinement converged cannot be established from this output, and a "
cloudra/tools/core/xtb_path_runner.py:611               "non-converged path must not be handed over as a TS guess"
cloudra/tools/core/xtb_path_runner.py:612           )
cloudra/tools/core/xtb_path_runner.py:613           result.message = (
cloudra/tools/core/xtb_path_runner.py:614               f"the xTB path search reported no path-refinement termination line, so convergence "
cloudra/tools/core/xtb_path_runner.py:615               f"could not be confirmed. Full log: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:616           )
cloudra/tools/core/xtb_path_runner.py:617           return result
cloudra/tools/core/xtb_path_runner.py:618
cloudra/tools/core/xtb_path_runner.py:619       result.termination_reason = "completed"
cloudra/tools/core/xtb_path_runner.py:620       result.usable_as_guess = True
cloudra/tools/core/xtb_path_runner.py:621       result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:622           "path-frame comment energies are RELATIVE kcal/mol; the TS candidate's comment energy is "
cloudra/tools/core/xtb_path_runner.py:623           "ABSOLUTE Eh — the two are reported in separate fields on purpose "
cloudra/tools/core/xtb_path_runner.py:624           "(verified against single points on this machine)"
cloudra/tools/core/xtb_path_runner.py:625       )
cloudra/tools/core/xtb_path_runner.py:626       result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:627           "the TS candidate is a guess sampled by metadynamics, NOT a saddle point and NOT verified; "
cloudra/tools/core/xtb_path_runner.py:628           "frequency/IRC judgement still has to run"
cloudra/tools/core/xtb_path_runner.py:629       )
cloudra/tools/core/xtb_path_runner.py:630       result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:631           f"the engine declared the path refinement converged ({MARKER_PATH_CONVERGED!r}) within the "
cloudra/tools/core/xtb_path_runner.py:632           f"{request.refinement_runs}-run cap"
cloudra/tools/core/xtb_path_runner.py:633       )
cloudra/tools/core/xtb_path_runner.py:634       result.message = (
cloudra/tools/core/xtb_path_runner.py:635           f"xtb path search completed: {len(frames)} path frames and a TS candidate "
cloudra/tools/core/xtb_path_runner.py:636           f"({ts_energy:.6f} Eh). Run directory: {run_dir}"
cloudra/tools/core/xtb_path_runner.py:637       )
cloudra/tools/core/xtb_path_runner.py:638       return result
cloudra/tools/name_hints.py:1   import difflib
cloudra/tools/name_hints.py:2   from typing import Sequence
cloudra/tools/name_hints.py:3
cloudra/tools/name_hints.py:4   _RETIRED: dict[str, str] = {
cloudra/tools/name_hints.py:5       "text_reader": "read",
cloudra/tools/name_hints.py:6       "image_reader": "read",
cloudra/tools/name_hints.py:7       "pdf2md": "read",
cloudra/tools/name_hints.py:8       "pdf2image": "read",
cloudra/tools/name_hints.py:9       "read_file": "read",
cloudra/tools/name_hints.py:10      "web_search": "search (source=web)",
cloudra/tools/name_hints.py:11      "paper_searcher": "search (source=arxiv)",
cloudra/tools/name_hints.py:12      "openalex_searcher": "search (source=openalex)",
cloudra/tools/name_hints.py:13      "bashtools": "bash",
cloudra/tools/name_hints.py:14      "shell": "bash",
cloudra/tools/name_hints.py:15      "text_writer": "write",
cloudra/tools/name_hints.py:16      "write_file": "write",
cloudra/tools/name_hints.py:17      "edit_file": "edit",
cloudra/tools/name_hints.py:18      "str_replace": "edit",
cloudra/tools/name_hints.py:19      "getskill": "get_skill",
```

```
cloudra/tools/name_hints.py:20      "web_fetcher": "browser",
cloudra/tools/name_hints.py:21      "web_fetch": "browser",
cloudra/tools/name_hints.py:22      "headless_browser": "browser",
cloudra/tools/name_hints.py:23      "SubAgentLauncher": "subagent_launcher",
cloudra/tools/name_hints.py:24      "SubAgentSwarm": "subagent_swarm",
cloudra/tools/name_hints.py:25  }
cloudra/tools/name_hints.py:26
cloudra/tools/name_hints.py:27
cloudra/tools/name_hints.py:28  def hint_unknown_tool(name: str, available_tools: Sequence[str] | None = None) -> str:
cloudra/tools/name_hints.py:29      target = _RETIRED.get(name)
cloudra/tools/name_hints.py:30      if target:
cloudra/tools/name_hints.py:31          return (
cloudra/tools/name_hints.py:32              f"[Tool error] unknown tool {name!r}. "
cloudra/tools/name_hints.py:33              f"Use `{target}` instead."
cloudra/tools/name_hints.py:34          )
cloudra/tools/name_hints.py:35      if available_tools is None:
cloudra/tools/name_hints.py:36          try:
cloudra/tools/name_hints.py:37              from cloudra.tools import get_tools_lst
cloudra/tools/name_hints.py:38              available_tools = [e.tool_name for e in get_tools_lst()]
cloudra/tools/name_hints.py:39          except Exception:
cloudra/tools/name_hints.py:40              available_tools = []
cloudra/tools/name_hints.py:41      matches = difflib.get_close_matches(name, available_tools, n=2, cutoff=0.35)
cloudra/tools/name_hints.py:42      if matches:
cloudra/tools/name_hints.py:43          return (
cloudra/tools/name_hints.py:44              f"[Tool error] unknown tool {name!r}. "
cloudra/tools/name_hints.py:45              f"Did you mean '{matches[0]}'? Choose an available tool and retry."
cloudra/tools/name_hints.py:46          )
cloudra/tools/name_hints.py:47      return (
cloudra/tools/name_hints.py:48          f"[Tool error] unknown tool {name!r}. "
cloudra/tools/name_hints.py:49          "Choose an available tool and retry."
cloudra/tools/name_hints.py:50      )
cloudra/tools/name_hints.py:51
cloudra/tools/name_hints.py:52
cloudra/tools/name_hints.py:53  def is_retired_tool_name(name: str) -> bool:
cloudra/tools/name_hints.py:54      return name in _RETIRED
cloudra/tools/registry.py:1 """Explicit built-in tool manifest.
cloudra/tools/registry.py:2
cloudra/tools/registry.py:3 Keep this module dependency-free: the execution kernel imports
cloudra/tools/registry.py:4 ``cloudra.tools``, while tool implementations pull in optional extras
cloudra/tools/registry.py:5 (chem/visual) and heavy modules that may legitimately be missing.  The registry
cloudra/tools/registry.py:6 therefore stores import paths instead of importing tool classes at module
cloudra/tools/registry.py:7 import time.
cloudra/tools/registry.py:8
cloudra/tools/registry.py:9 Optional semantic fields are the single source of truth for downstream policy
cloudra/tools/registry.py:10    projections (DAG gate / runtime profiles / evidence derivation).  Downstream
cloudra/tools/registry.py:11    modules must query :mod:`cloudra.tools.capabilities` instead of maintaining
cloudra/tools/registry.py:12    their own per-tool name sets.
cloudra/tools/registry.py:13    """
cloudra/tools/registry.py:14
cloudra/tools/registry.py:15    from dataclasses import dataclass
cloudra/tools/registry.py:16
cloudra/tools/registry.py:17
```

```
cloudra/tools/registry.py:18    @dataclass(frozen=True)
cloudra/tools/registry.py:19    class ToolRegistration:
cloudra/tools/registry.py:20        module: str
cloudra/tools/registry.py:21        class_name: str
cloudra/tools/registry.py:22
cloudra/tools/registry.py:23        # Declared wire name. Only needed when the tool has non-default semantics
cloudra/tools/registry.py:24        # (or a tool_name that differs from class_name); capabilities only projects
cloudra/tools/registry.py:25        # entries that carry it.
cloudra/tools/registry.py:26        tool_name: str = ""
cloudra/tools/registry.py:27
cloudra/tools/registry.py:28        # DAG semantics. All flags default to the conservative/plain behavior.
cloudra/tools/registry.py:29        dag_observe: bool = True
cloudra/tools/registry.py:30        evidence_deriver: str = ""  # "module:function(current, role, arguments, result)->dict"
cloudra/tools/registry.py:31        # 事实适配器（contract_version=2）："module:function(payload, source_ref)
cloudra/tools/registry.py:32        # -> list[EvidenceFact]"；从已登记证据中抽类型化事实（含反证）。
cloudra/tools/registry.py:33        dag_fact_adapter: str = ""
cloudra/tools/registry.py:34        # 证据产生工具：完成的调用产出计算证据（知识检索触发语义）。
cloudra/tools/registry.py:35        # §5 起不再有"自动入 CallReview 收件箱"的开关——送审只能显式
cloudra/tools/registry.py:36        # （dag_bind_evidence with claim）。
cloudra/tools/registry.py:37        dag_evidence_producer: bool = False
cloudra/tools/registry.py:38        # Phase 3 门槛：mechanism 未被接受前拦截的计算入口（开算/写计算输入）。
cloudra/tools/registry.py:39        dag_compute_entry: bool = False
cloudra/tools/registry.py:40        dag_compute_predicate: str = ""  # "module:function(arguments)->bool", overrides
    dag_compute_entry
cloudra/tools/registry.py:41
cloudra/tools/registry.py:42        # Runtime availability. The default keeps the Android allow-list opt-in.
cloudra/tools/registry.py:43        runtime_profiles: frozenset[str] = frozenset({"desktop"})
cloudra/tools/registry.py:44
cloudra/tools/registry.py:45        # Visual-mode gate: image-producing/consuming tools hidden unless the
cloudra/tools/registry.py:46        # agent's visual_mode setting is on.
cloudra/tools/registry.py:47        visual: bool = False
cloudra/tools/registry.py:48
cloudra/tools/registry.py:49        # Supervisor capability: whether the scientific Supervisor may discover and
cloudra/tools/registry.py:50        # execute this tool. One of: "none" (default, hidden), "evidence_read"
cloudra/tools/registry.py:51        # (read-only evidence access), "skill_read", "hub" (AgentHub, action-gated),
cloudra/tools/registry.py:52        # "review_decision" (control-plane verdicts). Projected exclusively via
cloudra/tools/registry.py:53        # tools/capabilities.py; do not keep separate supervisor tool name sets.
cloudra/tools/registry.py:54        supervisor_access: str = "none"
cloudra/tools/registry.py:55
cloudra/tools/registry.py:56        # Invocation routing. Direct controls stay top-level wire tools in the
cloudra/tools/registry.py:57        # javascript protocol (exposed next to js_exec) and are rejected inside a
cloudra/tools/registry.py:58        # guest program; ordinary business tools are the only guest-callable ones.
cloudra/tools/registry.py:59        invocation_direct_only: bool = False
cloudra/tools/registry.py:60
cloudra/tools/registry.py:61        # Config gate: "<section>.<field>" naming a boolean on CloudraConfig.
cloudra/tools/registry.py:62        # When the flag is false the tool is hidden from the wire projection and
cloudra/tools/registry.py:63        # hard-fails at call time (the tool itself must check the flag too).
cloudra/tools/registry.py:64        config_gate: str = ""
cloudra/tools/registry.py:65
cloudra/tools/registry.py:66        # Role allowlist: non-empty restricts the tool to the listed role_ids —
cloudra/tools/registry.py:67        # the wire projection hides it from other roles and PermissionStage
cloudra/tools/registry.py:68        # hard-fails their calls. Empty means every role may use it (default).
cloudra/tools/registry.py:69        # Never set on direct controls, dag_* or supervisor-surface tools: their
```

```
cloudra/tools/registry.py:70        # gates (invocation_direct_only / orchestration mode / supervisor_access)
cloudra/tools/registry.py:71        # are separate mechanisms.
cloudra/tools/registry.py:72        role_allowlist: frozenset[str] = frozenset()
cloudra/tools/registry.py:73
cloudra/tools/registry.py:74
cloudra/tools/registry.py:75    # 化学/QM 工具的默认使用者：科学计算角色 + Coordinator（benchmark 题面
cloudra/tools/registry.py:76    # agent 以 coordinator 角色运行，绝不能失去化学工具链）。supervisor 在列
cloudra/tools/registry.py:77    # 仅为保住其 evidence_read 投影面（log_parser/reaction_mapping）——它的
cloudra/tools/registry.py:78    # 实际可见面仍由 supervisor_access 收窄，allowlist 对它是只减不增。
cloudra/tools/registry.py:79    _SCIENCE_ROLES = frozenset(
cloudra/tools/registry.py:80        {"coordinator", "small_mol", "md", "dft", "experiment", "supervisor"}
cloudra/tools/registry.py:81    )
cloudra/tools/registry.py:82
cloudra/tools/registry.py:83    TOOL_REGISTRY: tuple[ToolRegistration, ...] = (
cloudra/tools/registry.py:84        ToolRegistration(
cloudra/tools/registry.py:85            module='cloudra.tools.Bashtool.bash',
cloudra/tools/registry.py:86            class_name='Bash',
cloudra/tools/registry.py:87            tool_name='bash',
cloudra/tools/registry.py:88            dag_compute_predicate='cloudra.tools.core.bash_habit_guard:
    is_substantive_bash_command',
cloudra/tools/registry.py:89        ),
cloudra/tools/registry.py:90        ToolRegistration(
cloudra/tools/registry.py:91            module='cloudra.tools.Browsertool.browser',
cloudra/tools/registry.py:92            class_name='HeadlessBrowser',
cloudra/tools/registry.py:93            tool_name='browser',
cloudra/tools/registry.py:94        ),
cloudra/tools/registry.py:95        ToolRegistration(
cloudra/tools/registry.py:96            module='cloudra.tools.Chemtool.conformer_search',
cloudra/tools/registry.py:97            class_name='ConformerSearch',
cloudra/tools/registry.py:98            tool_name='conformer_search',
cloudra/tools/registry.py:99            dag_compute_entry=True,
cloudra/tools/registry.py:100           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:101       ),
cloudra/tools/registry.py:102       ToolRegistration(
cloudra/tools/registry.py:103           module='cloudra.tools.Chemtool.electronic_analysis',
cloudra/tools/registry.py:104           class_name='ElectronicAnalysis',
cloudra/tools/registry.py:105           tool_name='electronic_analysis',
cloudra/tools/registry.py:106           dag_compute_entry=True,
cloudra/tools/registry.py:107           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:108       ),
cloudra/tools/registry.py:109       ToolRegistration(
cloudra/tools/registry.py:110           module='cloudra.tools.Chemtool.format_converter',
cloudra/tools/registry.py:111           class_name='FormatConverter',
cloudra/tools/registry.py:112           tool_name='format_converter',
cloudra/tools/registry.py:113           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:114       ),
cloudra/tools/registry.py:115       ToolRegistration(
cloudra/tools/registry.py:116           module='cloudra.tools.Chemtool.qm_input_writer',
cloudra/tools/registry.py:117           class_name='QmInputWriter',
cloudra/tools/registry.py:118           tool_name='qm_input_writer',
cloudra/tools/registry.py:119           dag_compute_entry=True,
cloudra/tools/registry.py:120           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:121       ),
```

```
cloudra/tools/registry.py:122       ToolRegistration(
cloudra/tools/registry.py:123           module='cloudra.tools.Chemtool.image_to_smiles',
cloudra/tools/registry.py:124           class_name='ImageToSmiles',
cloudra/tools/registry.py:125           tool_name='image_to_smiles',
cloudra/tools/registry.py:126           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:127       ),
cloudra/tools/registry.py:128       ToolRegistration(
cloudra/tools/registry.py:129           module='cloudra.tools.Chemtool.cdxml_to_graph',
cloudra/tools/registry.py:130           class_name='CdxmlToGraph',
cloudra/tools/registry.py:131           tool_name='cdxml_to_graph',
cloudra/tools/registry.py:132           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:133       ),
cloudra/tools/registry.py:134       ToolRegistration(
cloudra/tools/registry.py:135           module='cloudra.tools.Chemtool.internal_coordinates',
cloudra/tools/registry.py:136           class_name='InternalCoordinates',
cloudra/tools/registry.py:137           tool_name='internal_coordinates',
cloudra/tools/registry.py:138           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:139       ),
cloudra/tools/registry.py:140       ToolRegistration(
cloudra/tools/registry.py:141           module='cloudra.tools.Chemtool.ir_processor',
cloudra/tools/registry.py:142           class_name='IRProcessor',
cloudra/tools/registry.py:143           tool_name='ir_processor',
cloudra/tools/registry.py:144           # 实验谱图处理：experiment 角色专属
cloudra/tools/registry.py:145           role_allowlist=frozenset({'experiment'}),
cloudra/tools/registry.py:146       ),
cloudra/tools/registry.py:147       ToolRegistration(
cloudra/tools/registry.py:148           module='cloudra.tools.Chemtool.log_parser',
cloudra/tools/registry.py:149           class_name='LogParser',
cloudra/tools/registry.py:150           tool_name='log_parser',
cloudra/tools/registry.py:151           evidence_deriver='cloudra.tools.Chemtool.log_parser:derive_delivery_changes',
cloudra/tools/registry.py:152           dag_fact_adapter='cloudra.dag.fact_adapters:log_parser_facts',
cloudra/tools/registry.py:153           dag_evidence_producer=True,
cloudra/tools/registry.py:154           supervisor_access='evidence_read',
cloudra/tools/registry.py:155           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:156       ),
cloudra/tools/registry.py:157       ToolRegistration(
cloudra/tools/registry.py:158           module='cloudra.tools.Chemtool.molecule_drawer',
cloudra/tools/registry.py:159           class_name='MoleculeDrawer',
cloudra/tools/registry.py:160           tool_name='molecule_drawer',
cloudra/tools/registry.py:161           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:162       ),
cloudra/tools/registry.py:163       ToolRegistration(
cloudra/tools/registry.py:164           module='cloudra.tools.Chemtool.mechanism_predict',
cloudra/tools/registry.py:165           class_name='MechanismPredict',
cloudra/tools/registry.py:166           tool_name='mechanism_predict',
cloudra/tools/registry.py:167           # ML 机理假设生成器（FlowER 外部 conda env）：不是计算证据；
cloudra/tools/registry.py:168           # flower.enabled 默认开（未装环境时调用层引导 setup_flower.sh）。
cloudra/tools/registry.py:169           config_gate='flower.enabled',
cloudra/tools/registry.py:170           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:171       ),
cloudra/tools/registry.py:172       ToolRegistration(
cloudra/tools/registry.py:173           module='cloudra.tools.Chemtool.name_to_structure',
```

```
cloudra/tools/registry.py:174           class_name='NameToStructure',
cloudra/tools/registry.py:175           tool_name='name_to_structure',
cloudra/tools/registry.py:176           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:177       ),
cloudra/tools/registry.py:178       ToolRegistration(
cloudra/tools/registry.py:179           module='cloudra.tools.Chemtool.nmr_processor',
cloudra/tools/registry.py:180           class_name='NMRProcessor',
cloudra/tools/registry.py:181           tool_name='nmr_processor',
cloudra/tools/registry.py:182           # 实验谱图处理：experiment 角色专属
cloudra/tools/registry.py:183           role_allowlist=frozenset({'experiment'}),
cloudra/tools/registry.py:184       ),
cloudra/tools/registry.py:185       ToolRegistration(
cloudra/tools/registry.py:186           module='cloudra.tools.Chemtool.pymol_render',
cloudra/tools/registry.py:187           class_name='PymolRender',
cloudra/tools/registry.py:188           tool_name='pymol_render',
cloudra/tools/registry.py:189           visual=True,
cloudra/tools/registry.py:190           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:191       ),
cloudra/tools/registry.py:192       ToolRegistration(
cloudra/tools/registry.py:193           module='cloudra.tools.Chemtool.query_compute_job',
cloudra/tools/registry.py:194           class_name='QueryComputeJob',
cloudra/tools/registry.py:195           tool_name='query_compute_job',
cloudra/tools/registry.py:196           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:197       ),
cloudra/tools/registry.py:198       ToolRegistration(
cloudra/tools/registry.py:199           module='cloudra.tools.Chemtool.query_compute_queue',
cloudra/tools/registry.py:200           class_name='QueryComputeQueue',
cloudra/tools/registry.py:201           tool_name='query_compute_queue',
cloudra/tools/registry.py:202           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:203       ),
cloudra/tools/registry.py:204       ToolRegistration(
cloudra/tools/registry.py:205           module='cloudra.tools.Chemtool.mark_compute_job_failed',
cloudra/tools/registry.py:206           class_name='MarkComputeJobFailed',
cloudra/tools/registry.py:207           tool_name='mark_compute_job_failed',
cloudra/tools/registry.py:208           dag_observe=False,
cloudra/tools/registry.py:209           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:210       ),
cloudra/tools/registry.py:211       ToolRegistration(
cloudra/tools/registry.py:212           module='cloudra.tools.Chemtool.reaction_mapping',
cloudra/tools/registry.py:213           class_name='ReactionMapping',
cloudra/tools/registry.py:214           tool_name='reaction_mapping',
cloudra/tools/registry.py:215           evidence_deriver='cloudra.tools.Chemtool.reaction_mapping:derive_delivery_changes',
cloudra/tools/registry.py:216           dag_fact_adapter='cloudra.dag.fact_adapters:reaction_mapping_facts',
cloudra/tools/registry.py:217           dag_evidence_producer=True,
cloudra/tools/registry.py:218           supervisor_access='evidence_read',
cloudra/tools/registry.py:219           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:220       ),
cloudra/tools/registry.py:221       # TS 搜索 SOP 的 agent 可见接口（只有这两个函数：刻意没有 status/查询接口）
cloudra/tools/registry.py:222       ToolRegistration(
cloudra/tools/registry.py:223           'cloudra.tools.Chemtool.ts_search_process', 'TsPrepare', tool_name='ts_prepare',
cloudra/tools/registry.py:224           role_allowlist=_SCIENCE_ROLES,
cloudra/tools/registry.py:225       ),
```

```
        ToolRegistration(
            'cloudra.tools.Chemtool.ts_search_process', 'TsResume', tool_name='ts_resume',
            role_allowlist=_SCIENCE_ROLES,
        ),
        ToolRegistration(
            module='cloudra.tools.Chemtool.smiles_to_mol',
            class_name='SmilesToMol',
            tool_name='smiles_to_mol',
            role_allowlist=_SCIENCE_ROLES,
        ),
        ToolRegistration(
            module='cloudra.tools.Chemtool.submit_compute_job',
            class_name='SubmitComputeJob',
            tool_name='submit_compute_job',
            dag_compute_entry=True,
            role_allowlist=_SCIENCE_ROLES,
        ),
        ToolRegistration(
            module='cloudra.tools.Chemtool.thermo_analysis',
            class_name='ThermoAnalysis',
            tool_name='thermo_analysis',
            evidence_deriver='cloudra.tools.Chemtool.thermo_analysis:derive_delivery_changes',
            dag_evidence_producer=True,
            role_allowlist=_SCIENCE_ROLES,
        ),
        ToolRegistration(
            module='cloudra.tools.Chemtool.ts_guess',
            class_name='TsGuess',
            tool_name='ts_guess',
            evidence_deriver='cloudra.tools.Chemtool.ts_guess:derive_delivery_changes',
            dag_fact_adapter='cloudra.dag.fact_adapters:ts_guess_facts',
            dag_evidence_producer=True,
            dag_compute_predicate='cloudra.tools.Chemtool.ts_guess:is_compute_entry_call',
            role_allowlist=_SCIENCE_ROLES,
        ),
        ToolRegistration(
            module='cloudra.tools.Interactool.ask_user',
            class_name='AskUser',
            tool_name='ask_user',
            invocation_direct_only=True,
        ),
        ToolRegistration(
            module='cloudra.tools.Interactool.email_reader',
            class_name='EmailReader',
            tool_name='email_reader',
            config_gate='email.enabled',
        ),
        ToolRegistration(
            module='cloudra.tools.Interactool.email_sender',
            class_name='EmailSender',
            tool_name='email_sender',
            config_gate='email.enabled',
```

```
cloudra/tools/registry.py:278       ),
cloudra/tools/registry.py:279       ToolRegistration(
cloudra/tools/registry.py:280           module='cloudra.tools.Managetool.agent_hub',
cloudra/tools/registry.py:281           class_name='AgentHub',
cloudra/tools/registry.py:282           tool_name='agent_hub',
cloudra/tools/registry.py:283           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:284           supervisor_access='hub',
cloudra/tools/registry.py:285       ),
cloudra/tools/registry.py:286       ToolRegistration(
cloudra/tools/registry.py:287           module='cloudra.tools.Managetool.get_subagent_report',
cloudra/tools/registry.py:288           class_name='GetSubAgentReport',
cloudra/tools/registry.py:289           tool_name='get_subagent_report',
cloudra/tools/registry.py:290           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:291       ),
cloudra/tools/registry.py:292       ToolRegistration(
cloudra/tools/registry.py:293           module='cloudra.tools.Managetool.schedule',
cloudra/tools/registry.py:294           class_name='Schedule',
cloudra/tools/registry.py:295           tool_name='schedule',
cloudra/tools/registry.py:296           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:297           invocation_direct_only=True,
cloudra/tools/registry.py:298       ),
cloudra/tools/registry.py:299       ToolRegistration(
cloudra/tools/registry.py:300           module='cloudra.tools.Managetool.subagent_launcher',
cloudra/tools/registry.py:301           class_name='SubAgentLauncher',
cloudra/tools/registry.py:302           tool_name='subagent_launcher',
cloudra/tools/registry.py:303           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:304           role_allowlist=frozenset({'coordinator'}),
cloudra/tools/registry.py:305       ),
cloudra/tools/registry.py:306       ToolRegistration(
cloudra/tools/registry.py:307           module='cloudra.tools.Managetool.subagent_swarm',
cloudra/tools/registry.py:308           class_name='SubAgentSwarm',
cloudra/tools/registry.py:309           tool_name='subagent_swarm',
cloudra/tools/registry.py:310           role_allowlist=frozenset({'coordinator'}),
cloudra/tools/registry.py:311       ),
cloudra/tools/registry.py:312       ToolRegistration('cloudra.tools.Plantool.goal', 'GoalSet'),
cloudra/tools/registry.py:313       ToolRegistration(
cloudra/tools/registry.py:314           module='cloudra.tools.Plantool.dag_tools.dag_view',
cloudra/tools/registry.py:315           class_name='DagView',
cloudra/tools/registry.py:316           tool_name='dag_view',
cloudra/tools/registry.py:317           dag_observe=False,
cloudra/tools/registry.py:318           supervisor_access='evidence_read',
cloudra/tools/registry.py:319       ),
cloudra/tools/registry.py:320       ToolRegistration(
cloudra/tools/registry.py:321           module='cloudra.tools.Plantool.dag_tools.dag_delivery_report',
cloudra/tools/registry.py:322           class_name='DagDeliveryReport',
cloudra/tools/registry.py:323           tool_name='dag_delivery_report',
cloudra/tools/registry.py:324           dag_observe=False,
cloudra/tools/registry.py:325           supervisor_access='evidence_read',
cloudra/tools/registry.py:326       ),
cloudra/tools/registry.py:327       ToolRegistration(
cloudra/tools/registry.py:328           module='cloudra.tools.Plantool.dag_tools.dag_add_species',
cloudra/tools/registry.py:329           class_name='DagAddSpecies',
```

```
cloudra/tools/registry.py:330           tool_name='dag_add_species',
cloudra/tools/registry.py:331           dag_observe=False,
cloudra/tools/registry.py:332       ),
cloudra/tools/registry.py:333       ToolRegistration(
cloudra/tools/registry.py:334           module='cloudra.tools.Plantool.dag_tools.dag_add_step',
cloudra/tools/registry.py:335           class_name='DagAddStep',
cloudra/tools/registry.py:336           tool_name='dag_add_step',
cloudra/tools/registry.py:337           dag_observe=False,
cloudra/tools/registry.py:338       ),
cloudra/tools/registry.py:339       ToolRegistration(
cloudra/tools/registry.py:340           module='cloudra.tools.Plantool.dag_tools.dag_update_delivery_element',
cloudra/tools/registry.py:341           class_name='DagUpdateDeliveryElement',
cloudra/tools/registry.py:342           tool_name='dag_update_delivery_element',
cloudra/tools/registry.py:343           dag_observe=False,
cloudra/tools/registry.py:344       ),
cloudra/tools/registry.py:345       ToolRegistration(
cloudra/tools/registry.py:346           module='cloudra.tools.Plantool.dag_tools.dag_remove_edge',
cloudra/tools/registry.py:347           class_name='DagRemoveEdge',
cloudra/tools/registry.py:348           tool_name='dag_remove_edge',
cloudra/tools/registry.py:349           dag_observe=False,
cloudra/tools/registry.py:350       ),
cloudra/tools/registry.py:351       ToolRegistration(
cloudra/tools/registry.py:352           module='cloudra.tools.Plantool.dag_tools.dag_remove_node',
cloudra/tools/registry.py:353           class_name='DagRemoveNode',
cloudra/tools/registry.py:354           tool_name='dag_remove_node',
cloudra/tools/registry.py:355           dag_observe=False,
cloudra/tools/registry.py:356           # 无 supervisor_access：删图是研究侧（Coordinator/子 agent）的修改
cloudra/tools/registry.py:357           # 权限；Supervisor 保留审查职责、不顺带获得直接删图能力。
cloudra/tools/registry.py:358       ),
cloudra/tools/registry.py:359       ToolRegistration(
cloudra/tools/registry.py:360           module='cloudra.tools.Plantool.dag_tools.dag_bind_evidence',
cloudra/tools/registry.py:361           class_name='DagBindEvidence',
cloudra/tools/registry.py:362           tool_name='dag_bind_evidence',
cloudra/tools/registry.py:363           dag_observe=False,
cloudra/tools/registry.py:364       ),
cloudra/tools/registry.py:365       ToolRegistration(
cloudra/tools/registry.py:366           module='cloudra.tools.Plantool.dag_tools.dag_propose_mechanism',
cloudra/tools/registry.py:367           class_name='DagProposeMechanism',
cloudra/tools/registry.py:368           tool_name='dag_propose_mechanism',
cloudra/tools/registry.py:369           dag_observe=False,
cloudra/tools/registry.py:370       ),
cloudra/tools/registry.py:371       ToolRegistration(
cloudra/tools/registry.py:372           module='cloudra.tools.Plantool.dag_tools.dag_submit_step_evidence',
cloudra/tools/registry.py:373           class_name='DagSubmitStepEvidence',
cloudra/tools/registry.py:374           tool_name='dag_submit_step_evidence',
cloudra/tools/registry.py:375           dag_observe=False,
cloudra/tools/registry.py:376       ),
cloudra/tools/registry.py:377       ToolRegistration(
cloudra/tools/registry.py:378           module='cloudra.tools.Plantool.dag_tools.dag_submit_species_evidence',
cloudra/tools/registry.py:379           class_name='DagSubmitSpeciesEvidence',
cloudra/tools/registry.py:380           tool_name='dag_submit_species_evidence',
cloudra/tools/registry.py:381           dag_observe=False,
```

```
cloudra/tools/registry.py:382       ),
cloudra/tools/registry.py:383       ToolRegistration(
cloudra/tools/registry.py:384           module='cloudra.tools.Plantool.dag_tools.dag_review_step',
cloudra/tools/registry.py:385           class_name='DagReviewStep',
cloudra/tools/registry.py:386           tool_name='dag_review_step',
cloudra/tools/registry.py:387           dag_observe=False,
cloudra/tools/registry.py:388           supervisor_access='review_decision',
cloudra/tools/registry.py:389       ),
cloudra/tools/registry.py:390       ToolRegistration(
cloudra/tools/registry.py:391           module='cloudra.tools.Plantool.dag_tools.dag_review_species',
cloudra/tools/registry.py:392           class_name='DagReviewSpecies',
cloudra/tools/registry.py:393           tool_name='dag_review_species',
cloudra/tools/registry.py:394           dag_observe=False,
cloudra/tools/registry.py:395           supervisor_access='review_decision',
cloudra/tools/registry.py:396       ),
cloudra/tools/registry.py:397       ToolRegistration(
cloudra/tools/registry.py:398           module='cloudra.tools.Plantool.dag_tools.dag_accept_mechanism',
cloudra/tools/registry.py:399           class_name='DagAcceptMechanism',
cloudra/tools/registry.py:400           tool_name='dag_accept_mechanism',
cloudra/tools/registry.py:401           dag_observe=False,
cloudra/tools/registry.py:402           supervisor_access='review_decision',
cloudra/tools/registry.py:403       ),
cloudra/tools/registry.py:404       ToolRegistration(
cloudra/tools/registry.py:405           module='cloudra.tools.Plantool.dag_tools.dag_review_call',
cloudra/tools/registry.py:406           class_name='DagReviewCall',
cloudra/tools/registry.py:407           tool_name='dag_review_call',
cloudra/tools/registry.py:408           dag_observe=False,
cloudra/tools/registry.py:409           supervisor_access='review_decision',
cloudra/tools/registry.py:410       ),
cloudra/tools/registry.py:411       ToolRegistration(
cloudra/tools/registry.py:412           module='cloudra.tools.Plantool.dag_tools.dag_review_evidence',
cloudra/tools/registry.py:413           class_name='DagReviewEvidence',
cloudra/tools/registry.py:414           tool_name='dag_review_evidence',
cloudra/tools/registry.py:415           dag_observe=False,
cloudra/tools/registry.py:416           supervisor_access='review_decision',
cloudra/tools/registry.py:417       ),
cloudra/tools/registry.py:418       ToolRegistration(
cloudra/tools/registry.py:419           module='cloudra.tools.Plantool.dag_tools.dag_view_evidence',
cloudra/tools/registry.py:420           class_name='DagViewEvidence',
cloudra/tools/registry.py:421           tool_name='dag_view_evidence',
cloudra/tools/registry.py:422           dag_observe=False,
cloudra/tools/registry.py:423           supervisor_access='evidence_read',
cloudra/tools/registry.py:424       ),
cloudra/tools/registry.py:425       ToolRegistration(
cloudra/tools/registry.py:426           module='cloudra.tools.Plantool.dag_tools.dag_set_coverage_contract',
cloudra/tools/registry.py:427           class_name='DagSetCoverageContract',
cloudra/tools/registry.py:428           tool_name='dag_set_coverage_contract',
cloudra/tools/registry.py:429           dag_observe=False,
cloudra/tools/registry.py:430           supervisor_access='review_decision',
cloudra/tools/registry.py:431       ),
cloudra/tools/registry.py:432       ToolRegistration(
cloudra/tools/registry.py:433           module='cloudra.tools.Plantool.dag_tools.dag_message_supervisor',
```

```
cloudra/tools/registry.py:434            class_name='DagMessageSupervisor',
cloudra/tools/registry.py:435            tool_name='dag_message_supervisor',
cloudra/tools/registry.py:436            dag_observe=False,
cloudra/tools/registry.py:437            # 无 supervisor_access：这是研究 Agent/Coordinator → Supervisor 的
cloudra/tools/registry.py:438            # 请求入口、工具本身硬拒 Supervisor 调用；督导的回复路径是
cloudra/tools/registry.py:439            # dag_review_* 裁决工具与运行结束报告。
cloudra/tools/registry.py:440        ),
cloudra/tools/registry.py:441        ToolRegistration('cloudra.tools.Plantool.todo', 'TodoWrite'),
cloudra/tools/registry.py:442        ToolRegistration(
cloudra/tools/registry.py:443            module='cloudra.tools.Searchtool.read',
cloudra/tools/registry.py:444            class_name='Read',
cloudra/tools/registry.py:445            tool_name='read',
cloudra/tools/registry.py:446            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:447            supervisor_access='evidence_read',
cloudra/tools/registry.py:448        ),
cloudra/tools/registry.py:449        ToolRegistration(
cloudra/tools/registry.py:450            module='cloudra.tools.Searchtool.search',
cloudra/tools/registry.py:451            class_name='Search',
cloudra/tools/registry.py:452            tool_name='search',
cloudra/tools/registry.py:453            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:454        ),
cloudra/tools/registry.py:455        ToolRegistration(
cloudra/tools/registry.py:456            module='cloudra.tools.Skilltool.get_skill',
cloudra/tools/registry.py:457            class_name='GetSkill',
cloudra/tools/registry.py:458            tool_name='get_skill',
cloudra/tools/registry.py:459            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:460            dag_observe=False,
cloudra/tools/registry.py:461            supervisor_access='skill_read',
cloudra/tools/registry.py:462        ),
cloudra/tools/registry.py:463        ToolRegistration(
cloudra/tools/registry.py:464            module='cloudra.tools.Stoptool.stop_tool',
cloudra/tools/registry.py:465            class_name='StopTool',
cloudra/tools/registry.py:466            tool_name='stop',
cloudra/tools/registry.py:467            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:468            invocation_direct_only=True,
cloudra/tools/registry.py:469        ),
cloudra/tools/registry.py:470        ToolRegistration(
cloudra/tools/registry.py:471            module='cloudra.tools.Writetool.edit',
cloudra/tools/registry.py:472            class_name='Edit',
cloudra/tools/registry.py:473            tool_name='edit',
cloudra/tools/registry.py:474            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:475        ),
cloudra/tools/registry.py:476        ToolRegistration(
cloudra/tools/registry.py:477            module='cloudra.tools.Writetool.write',
cloudra/tools/registry.py:478            class_name='Write',
cloudra/tools/registry.py:479            tool_name='write',
cloudra/tools/registry.py:480            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:481        ),
cloudra/tools/registry.py:482        ToolRegistration(
cloudra/tools/registry.py:483            module='cloudra.tools.Writetool.update_memory',
cloudra/tools/registry.py:484            class_name='UpdateMemory',
cloudra/tools/registry.py:485            tool_name='update_memory',
```

```
cloudra/tools/registry.py:486           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:487       ),
cloudra/tools/registry.py:488   )
cloudra/web_launcher.py:1   """Web frontend launcher used by the `cloudra` command.
cloudra/web_launcher.py:2
cloudra/web_launcher.py:3   Keeps Node/npm knowledge out of the CLI module.  The launcher works for source
cloudra/web_launcher.py:4   checkouts and editable installs; packaged wheels can point CLOUDRA_FRONTEND_DIR
cloudra/web_launcher.py:5   at an unpacked frontend directory.
cloudra/web_launcher.py:6   """
cloudra/web_launcher.py:7
cloudra/web_launcher.py:8   from __future__ import annotations
cloudra/web_launcher.py:9
cloudra/web_launcher.py:10  import logging
cloudra/web_launcher.py:11  import os
cloudra/web_launcher.py:12  import shutil
cloudra/web_launcher.py:13  import socket
cloudra/web_launcher.py:14  import subprocess
cloudra/web_launcher.py:15  import sys
cloudra/web_launcher.py:16  import time
cloudra/web_launcher.py:17  import urllib.request
cloudra/web_launcher.py:18  import webbrowser
cloudra/web_launcher.py:19  from pathlib import Path
cloudra/web_launcher.py:20
cloudra/web_launcher.py:21  from cloudra.services.daemon_supervisor import ensure_daemons
cloudra/web_launcher.py:22
cloudra/web_launcher.py:23  # 常驻 SSE 不会自己结束（SPA 一打开就挂着 EventSource，见 frontend/src/api.ts），
cloudra/web_launcher.py:24  # 而 uvicorn 默认（None）会**无限等待活跃连接**：`cloudra stop` 因此报「API: 已停止」
cloudra/web_launcher.py:25  # 却关不死进程（GH #38）。给一个有界上界，超时后由 uvicorn 取消这些连接——流生成器
cloudra/web_launcher.py:26  # 的 finally 照常退订。取值与本文件的 _terminate_runtime(grace=20) 对齐。
cloudra/web_launcher.py:27  _GRACEFUL_SHUTDOWN_TIMEOUT_SECONDS = 20
cloudra/web_launcher.py:28
cloudra/web_launcher.py:29  # 默认只绑回环。`--host` 可显式改绑（局域网/SSH 远程访问场景）；绑定地址经
cloudra/web_launcher.py:30  # CLOUDRA_API_BIND_HOST 传进 uvicorn 子进程，api_v2 的 Host 白名单据此放开
cloudra/web_launcher.py:31  # （见 app.py::_host_allowlist）。
cloudra/web_launcher.py:32  _DEFAULT_BIND_HOST = "127.0.0.1"
cloudra/web_launcher.py:33  _LOOPBACK_BIND_HOSTS = {"127.0.0.1", "localhost", "::1", "[::1]"}
cloudra/web_launcher.py:34  _WILDCARD_BIND_HOSTS = {"0.0.0.0", "::", "*"}
cloudra/web_launcher.py:35
cloudra/web_launcher.py:36  logger = logging.getLogger(__name__)
cloudra/web_launcher.py:37
cloudra/web_launcher.py:38
cloudra/web_launcher.py:39  def _frontend_dir() -> Path:
cloudra/web_launcher.py:40      override = os.environ.get("CLOUDRA_FRONTEND_DIR", "").strip()
cloudra/web_launcher.py:41      candidates = []
cloudra/web_launcher.py:42      if override:
cloudra/web_launcher.py:43          candidates.append(Path(override))
cloudra/web_launcher.py:44      # cloudra/web_launcher.py -> cloudra/.. == repository root (source checkout)
cloudra/web_launcher.py:45      candidates.append(Path(__file__).resolve().parents[1] / "frontend")
cloudra/web_launcher.py:46      candidates.append(Path.cwd() / "frontend")
cloudra/web_launcher.py:47      for candidate in candidates:
cloudra/web_launcher.py:48          if (candidate / "package.json").is_file():
cloudra/web_launcher.py:49              return candidate.resolve()
```

```
cloudra/web_launcher.py:50      raise RuntimeError(
cloudra/web_launcher.py:51          "frontend directory not found; set CLOUDRA_FRONTEND_DIR to the "
cloudra/web_launcher.py:52          "directory containing frontend/package.json"
cloudra/web_launcher.py:53      )
cloudra/web_launcher.py:54
cloudra/web_launcher.py:55
cloudra/web_launcher.py:56  def _check_node(frontend: Path) -> None:
cloudra/web_launcher.py:57      npm = shutil.which("npm")
cloudra/web_launcher.py:58      node = shutil.which("node")
cloudra/web_launcher.py:59      if npm is None or node is None:
cloudra/web_launcher.py:60          raise RuntimeError(
cloudra/web_launcher.py:61              "Node.js >= 18 and npm are required for the Cloudra web frontend. "
cloudra/web_launcher.py:62              f"Frontend found at {frontend}. Install from https://nodejs.org/"
cloudra/web_launcher.py:63          )
cloudra/web_launcher.py:64
cloudra/web_launcher.py:65
cloudra/web_launcher.py:66  def _npm_argv() -> list[str]:
cloudra/web_launcher.py:67      """npm as an invokable argv prefix (fully resolved executable)."""
cloudra/web_launcher.py:68      npm = shutil.which("npm")
cloudra/web_launcher.py:69      if npm is None:
cloudra/web_launcher.py:70          raise RuntimeError(
cloudra/web_launcher.py:71              "npm not found on PATH; install Node.js >= 18 or run with --skip-build"
cloudra/web_launcher.py:72          )
cloudra/web_launcher.py:73      return [npm]
cloudra/web_launcher.py:74
cloudra/web_launcher.py:75
cloudra/web_launcher.py:76  def _frontend_source_newer_than_dist(frontend: Path) -> bool:
cloudra/web_launcher.py:77      dist_index = frontend / "dist" / "index.html"
cloudra/web_launcher.py:78      if not dist_index.is_file():
cloudra/web_launcher.py:79          return True
cloudra/web_launcher.py:80      dist_mtime = dist_index.stat().st_mtime
cloudra/web_launcher.py:81      for root in (frontend / "src", frontend / "public"):
cloudra/web_launcher.py:82          if not root.is_dir():
cloudra/web_launcher.py:83              continue
cloudra/web_launcher.py:84          for path in root.rglob("*"):
cloudra/web_launcher.py:85              if path.is_file() and path.stat().st_mtime > dist_mtime:
cloudra/web_launcher.py:86                  return True
cloudra/web_launcher.py:87      for name in ("index.html", "vite.config.ts", "package.json"):
cloudra/web_launcher.py:88          path = frontend / name
cloudra/web_launcher.py:89          if path.is_file() and path.stat().st_mtime > dist_mtime:
cloudra/web_launcher.py:90              return True
cloudra/web_launcher.py:91      return False
cloudra/web_launcher.py:92
cloudra/web_launcher.py:93
cloudra/web_launcher.py:94  def _ensure_frontend(frontend: Path, *, rebuild: bool = False, skip_build: bool = False) ->
    None:
cloudra/web_launcher.py:95      dist = frontend / "dist"
cloudra/web_launcher.py:96      if skip_build:
cloudra/web_launcher.py:97          if not (dist / "index.html").is_file():
cloudra/web_launcher.py:98              raise RuntimeError("--skip-build was requested but frontend/dist is missing")
cloudra/web_launcher.py:99          return
cloudra/web_launcher.py:100     _check_node(frontend)
cloudra/web_launcher.py:101     if not (frontend / "node_modules").is_dir():
```

```
cloudra/web_launcher.py:102         print("[cloudra] installing frontend dependencies...", flush=True)
cloudra/web_launcher.py:103         subprocess.run(_npm_argv() + ["install", "--no-audit", "--no-fund"], cwd=frontend,
check=True)
cloudra/web_launcher.py:104     stale = _frontend_source_newer_than_dist(frontend)
cloudra/web_launcher.py:105     if rebuild or not (dist / "index.html").is_file() or stale:
cloudra/web_launcher.py:106         if stale and not rebuild and (dist / "index.html").is_file():
cloudra/web_launcher.py:107             print("[cloudra] frontend source changed — rebuilding...", flush=True)
cloudra/web_launcher.py:108         else:
cloudra/web_launcher.py:109             print("[cloudra] building frontend...", flush=True)
cloudra/web_launcher.py:110         subprocess.run(_npm_argv() + ["run", "build"], cwd=frontend, check=True)
cloudra/web_launcher.py:111
cloudra/web_launcher.py:112
cloudra/web_launcher.py:113 def _probe_host(bind_host: str) -> str:
cloudra/web_launcher.py:114     """本机侧探测/访问用的地址。
cloudra/web_launcher.py:115
cloudra/web_launcher.py:116     通配绑定（0.0.0.0/::）在回环上同样可达，用回环探测；绑具体网卡地址时
cloudra/web_launcher.py:117     回环上根本没有监听，必须用该地址本身。IPv6 字面量加方括号才能拼进 URL。
cloudra/web_launcher.py:118     """
cloudra/web_launcher.py:119     if bind_host in _WILDCARD_BIND_HOSTS:
cloudra/web_launcher.py:120         return "127.0.0.1"
cloudra/web_launcher.py:121     if ":" in bind_host:
cloudra/web_launcher.py:122         return f"[{bind_host}]"
cloudra/web_launcher.py:123     return bind_host
cloudra/web_launcher.py:124
cloudra/web_launcher.py:125
cloudra/web_launcher.py:126 def _is_loopback_bind(bind_host: str) -> bool:
cloudra/web_launcher.py:127     return bind_host in _LOOPBACK_BIND_HOSTS
cloudra/web_launcher.py:128
cloudra/web_launcher.py:129
cloudra/web_launcher.py:130 def _may_reuse_runtime(bind_host: str) -> bool:
cloudra/web_launcher.py:131     """能否复用已在跑的实例。
cloudra/web_launcher.py:132
cloudra/web_launcher.py:133     通配绑定不能：探测走回环，而回环探测分辨不出对端绑的是 127.0.0.1 还是
cloudra/web_launcher.py:134     0.0.0.0——复用会让 `--host 0.0.0.0` 静默失效（用户以为已对外可访问，
cloudra/web_launcher.py:135     实际只绑在回环上）。回环/具体地址的探测地址就是绑定地址本身，判定可信。
cloudra/web_launcher.py:136     """
cloudra/web_launcher.py:137     return bind_host not in _WILDCARD_BIND_HOSTS
cloudra/web_launcher.py:138
cloudra/web_launcher.py:139
cloudra/web_launcher.py:140 def _port_is_bindable(bind_host: str, preferred: int) -> bool:
cloudra/web_launcher.py:141     family = socket.AF_INET6 if ":" in bind_host else socket.AF_INET
cloudra/web_launcher.py:142     with socket.socket(family, socket.SOCK_STREAM) as sock:
cloudra/web_launcher.py:143         try:
cloudra/web_launcher.py:144             sock.bind((bind_host, preferred))
cloudra/web_launcher.py:145         except OSError:
cloudra/web_launcher.py:146             return False
cloudra/web_launcher.py:147         return True
cloudra/web_launcher.py:148
cloudra/web_launcher.py:149
cloudra/web_launcher.py:150 def _runtime_is_healthy(probe_host: str, port: int) -> bool:
cloudra/web_launcher.py:151     try:
cloudra/web_launcher.py:152         with urllib.request.urlopen(
cloudra/web_launcher.py:153             f"http://{probe_host}:{port}/api/v2/runtime/health", timeout=1.0
```

```
cloudra/web_launcher.py:154         ) as resp:
cloudra/web_launcher.py:155             return 200 <= resp.status < 300
cloudra/web_launcher.py:156     except Exception:
cloudra/web_launcher.py:157         return False
cloudra/web_launcher.py:158
cloudra/web_launcher.py:159
cloudra/web_launcher.py:160 def _runtime_serves_spa(probe_host: str, port: int) -> bool:
cloudra/web_launcher.py:161     try:
cloudra/web_launcher.py:162         with urllib.request.urlopen(f"http://{probe_host}:{port}/", timeout=1.0) as resp:
cloudra/web_launcher.py:163             body = resp.read(4096).decode("utf-8", "ignore")
cloudra/web_launcher.py:164             return 200 <= resp.status < 300 and '<div id="root"></div>' in body
cloudra/web_launcher.py:165     except Exception:
cloudra/web_launcher.py:166         return False
cloudra/web_launcher.py:167
cloudra/web_launcher.py:168
cloudra/web_launcher.py:169 def _runtime_present(probe_host: str, port: int) -> bool:
cloudra/web_launcher.py:170     """同端口既提供 API 又提供已构建 SPA，才算可复用的 Cloudra 实例。"""
cloudra/web_launcher.py:171     return _runtime_is_healthy(probe_host, port) and _runtime_serves_spa(probe_host, port)
cloudra/web_launcher.py:172
cloudra/web_launcher.py:173
cloudra/web_launcher.py:174 def _pinned_runtime(bind_host: str, port: int) -> tuple[int, bool]:
cloudra/web_launcher.py:175     """显式指定的端口：不扫描邻近端口，占用而不可复用时交给 uvicorn 在绑定
cloudra/web_launcher.py:176     阶段报错（而不是悄悄换端口）。"""
cloudra/web_launcher.py:177     if not _may_reuse_runtime(bind_host):
cloudra/web_launcher.py:178         return port, False
cloudra/web_launcher.py:179     return port, _runtime_present(_probe_host(bind_host), port)
cloudra/web_launcher.py:180
cloudra/web_launcher.py:181
cloudra/web_launcher.py:182 def _find_api_port(preferred: int, env_name: str, bind_host: str) -> tuple[int, bool]:
cloudra/web_launcher.py:183     """Return (port, reused_existing_runtime).
cloudra/web_launcher.py:184
cloudra/web_launcher.py:185     A runtime is only reused when it both serves the API and the built SPA on
cloudra/web_launcher.py:186     the same port; otherwise we start a fresh single-port runtime on a free
cloudra/web_launcher.py:187     port so the frontend never points at a stale or foreign server.
cloudra/web_launcher.py:188     """
cloudra/web_launcher.py:189     probe_host = _probe_host(bind_host)
cloudra/web_launcher.py:190     reuse = _may_reuse_runtime(bind_host)
cloudra/web_launcher.py:191     env_value = os.environ.get(env_name)
cloudra/web_launcher.py:192     if env_value:
cloudra/web_launcher.py:193         port = int(env_value)
cloudra/web_launcher.py:194         return port, reuse and _runtime_present(probe_host, port)
cloudra/web_launcher.py:195     if _port_is_bindable(bind_host, preferred):
cloudra/web_launcher.py:196         return preferred, False
cloudra/web_launcher.py:197     if reuse and _runtime_present(probe_host, preferred):
cloudra/web_launcher.py:198         return preferred, True
cloudra/web_launcher.py:199     for port in range(preferred + 1, preferred + 100):
cloudra/web_launcher.py:200         if _port_is_bindable(bind_host, port):
cloudra/web_launcher.py:201             return port, False
cloudra/web_launcher.py:202     raise RuntimeError(f"no free port near {preferred}; set {env_name} explicitly")
cloudra/web_launcher.py:203
cloudra/web_launcher.py:204
cloudra/web_launcher.py:205 def _wait_healthy(url: str, timeout: float = 30.0) -> None:
```

```
    deadline = time.monotonic() + timeout
    while time.monotonic() < deadline:
        try:
            with urllib.request.urlopen(url, timeout=1.0) as resp:
                if 200 <= resp.status < 300:
                    return
        except Exception:
            pass
        time.sleep(0.25)
    raise RuntimeError(f"runtime did not become healthy at {url}")


def _terminate_runtime(runtime: subprocess.Popen, *, grace: float = 20.0) -> None:
    """SIGTERM → 有界等待 → SIGKILL 升级 → reap。

    SIGTERM 让 uvicorn 走优雅关停（lifespan 收尾：停准入 → 取消推理/收尾
    任务 → 关流/客户端 → 回收 JS host）；不许只发信号假定已回收（停止
    语义契约 6）。
    """
    if runtime.poll() is not None:
        return
    runtime.terminate()
    try:
        runtime.wait(timeout=grace)
        return
    except subprocess.TimeoutExpired:
        logger.warning("web runtime did not exit within %.0fs of SIGTERM; killing", grace)
    runtime.kill()
    try:
        runtime.wait(timeout=5)
    except subprocess.TimeoutExpired:
        logger.error("web runtime still alive after SIGKILL")


def _primary_address() -> str | None:
    """本机对外的首选 IPv4；取不到（无默认路由）返回 ``None``。

    UDP connect 只让内核选出口网卡，不实际发包；不解析 `ifconfig` 文本。
    仅用于把「别的机器能访问的地址」打印给用户——通配绑定下控制台里的
    回环 URL 复制到另一台机器上没有意义。
    """
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
            # 192.0.2.0/24 是 TEST-NET-1，永不路由到真实主机。
            sock.connect(("192.0.2.1", 1))
            return sock.getsockname()[0]
    except OSError:
        return None


def _ui_url(url_host: str, api_port: int, token: str | None) -> str:
    """访问 URL；认证开启时把 token 放 fragment 里（fragment 不进 HTTP 请求，
```

```
cloudra/web_launcher.py:258     前端启动时读入 sessionStorage 后从地址栏抹掉）。"""
cloudra/web_launcher.py:259     url = f"http://{url_host}:{api_port}"
cloudra/web_launcher.py:260     return f"{url}/#token={token}" if token is not None else url
cloudra/web_launcher.py:261
cloudra/web_launcher.py:262
cloudra/web_launcher.py:263 def _uvicorn_argv(api_port: int, bind_host: str) -> list[str]:
cloudra/web_launcher.py:264     """uvicorn 子进程命令行；关停上界见 ``_GRACEFUL_SHUTDOWN_TIMEOUT_SECONDS``。"""
cloudra/web_launcher.py:265     return [
cloudra/web_launcher.py:266         sys.executable,
cloudra/web_launcher.py:267         "-m",
cloudra/web_launcher.py:268         "uvicorn",
cloudra/web_launcher.py:269         "cloudra.services.api_v2:create_default_v2_app",
cloudra/web_launcher.py:270         "--factory",
cloudra/web_launcher.py:271         "--host",
cloudra/web_launcher.py:272         bind_host,
cloudra/web_launcher.py:273         "--port",
cloudra/web_launcher.py:274         str(api_port),
cloudra/web_launcher.py:275         "--timeout-graceful-shutdown",
cloudra/web_launcher.py:276         str(_GRACEFUL_SHUTDOWN_TIMEOUT_SECONDS),
cloudra/web_launcher.py:277         "--log-level",
cloudra/web_launcher.py:278         "warning",
cloudra/web_launcher.py:279     ]
cloudra/web_launcher.py:280
cloudra/web_launcher.py:281
cloudra/web_launcher.py:282 def _ensure_fd_limit(min_limit: int = 10240) -> None:
cloudra/web_launcher.py:283     """在 POSIX/macOS 上提升文件描述符软限制，防止高并发或长驻服务耗尽 fd。"""
cloudra/web_launcher.py:284     try:
cloudra/web_launcher.py:285         import resource
cloudra/web_launcher.py:286
cloudra/web_launcher.py:287         soft, hard = resource.getrlimit(resource.RLIMIT_NOFILE)
cloudra/web_launcher.py:288         if soft < min_limit:
cloudra/web_launcher.py:289             target = min(min_limit, hard) if hard != resource.RLIM_INFINITY else min_limit
cloudra/web_launcher.py:290             if target > soft:
cloudra/web_launcher.py:291                 resource.setrlimit(resource.RLIMIT_NOFILE, (target, hard))
cloudra/web_launcher.py:292     except Exception:
cloudra/web_launcher.py:293         pass
cloudra/web_launcher.py:294
cloudra/web_launcher.py:295
cloudra/web_launcher.py:296 def launch_browser(
cloudra/web_launcher.py:297     frontend: Path,
cloudra/web_launcher.py:298     *,
cloudra/web_launcher.py:299     rebuild: bool,
cloudra/web_launcher.py:300     skip_build: bool,
cloudra/web_launcher.py:301     host: str | None = None,
cloudra/web_launcher.py:302     port: int | None = None,
cloudra/web_launcher.py:303 ) -> int:
cloudra/web_launcher.py:304     _ensure_fd_limit()
cloudra/web_launcher.py:305     bind_host = (host or "").strip() or _DEFAULT_BIND_HOST
cloudra/web_launcher.py:306     from cloudra.services.api_token import require_token_for_bind
cloudra/web_launcher.py:307
cloudra/web_launcher.py:308     require_token_for_bind(bind_host)
cloudra/web_launcher.py:309     probe_host = _probe_host(bind_host)
```

```
cloudra/web_launcher.py:310     if port is not None:
cloudra/web_launcher.py:311         api_port, reused_runtime = _pinned_runtime(bind_host, port)
cloudra/web_launcher.py:312     else:
cloudra/web_launcher.py:313         api_port, reused_runtime = _find_api_port(8000, "CLOUDRA_API_PORT", bind_host)
cloudra/web_launcher.py:314     if not _is_loopback_bind(bind_host):
cloudra/web_launcher.py:315         print(
cloudra/web_launcher.py:316             f"[cloudra] warning: binding to non-loopback address {bind_host} — the API is "
cloudra/web_launcher.py:317             "reachable from the network and the bearer token is the only access control",
cloudra/web_launcher.py:318             flush=True,
cloudra/web_launcher.py:319         )
cloudra/web_launcher.py:320     _ensure_frontend(frontend, rebuild=rebuild, skip_build=skip_build)
cloudra/web_launcher.py:321     # 显式启动清除 `cloudra stop` 的持久关停 marker，否则 ensure 拒绝拉起。
cloudra/web_launcher.py:322     from cloudra.services.daemon_supervisor import clear_daemons_shutdown
cloudra/web_launcher.py:323
cloudra/web_launcher.py:324     clear_daemons_shutdown()
cloudra/web_launcher.py:325     ensure_daemons()
cloudra/web_launcher.py:326     from cloudra.persistence import paths
cloudra/web_launcher.py:327
cloudra/web_launcher.py:328     store_path = os.environ.get(
cloudra/web_launcher.py:329         "CLOUDRA_RUNTIME_STORE",
cloudra/web_launcher.py:330         str(paths.runtime_db_path()),
cloudra/web_launcher.py:331     )
cloudra/web_launcher.py:332     env = os.environ.copy()
cloudra/web_launcher.py:333     existing_pythonpath = env.get("PYTHONPATH", "")
cloudra/web_launcher.py:334     repo_root = str(Path(__file__).resolve().parents[1])
cloudra/web_launcher.py:335     env["PYTHONPATH"] = f"{repo_root}:{existing_pythonpath}" if existing_pythonpath else
    repo_root
cloudra/web_launcher.py:336     env["CLOUDRA_RUNTIME_STORE"] = store_path
cloudra/web_launcher.py:337     env["CLOUDRA_FRONTEND_SERVE_DIR"] = str(frontend / "dist")
cloudra/web_launcher.py:338     # 绑定地址传给 app（Host 校验白名单据此放开；绑回环时保持原白名单）。
cloudra/web_launcher.py:339     env["CLOUDRA_API_BIND_HOST"] = bind_host
cloudra/web_launcher.py:340     runtime = None
cloudra/web_launcher.py:341     if reused_runtime:
cloudra/web_launcher.py:342         print(f"[cloudra] reusing existing Cloudra runtime on port {api_port}", flush=True)
cloudra/web_launcher.py:343     else:
cloudra/web_launcher.py:344         print(f"[cloudra] starting web server on {bind_host}:{api_port}...", flush=True)
cloudra/web_launcher.py:345         runtime = subprocess.Popen(
cloudra/web_launcher.py:346             _uvicorn_argv(api_port, bind_host),
cloudra/web_launcher.py:347             env=env,
cloudra/web_launcher.py:348             # 固定子进程 cwd：`python -m` 会把 cwd 放到 sys.path 最前面，
cloudra/web_launcher.py:349             # cwd 下一个无 __init__.py 的 `cloudra` 目录会把已安装包遮蔽成
cloudra/web_launcher.py:350             # namespace 包（__version__ 缺失、子模块却能被 editable finder
cloudra/web_launcher.py:351             # 加载，报 "unknown location"）。frontend 目录已校验存在且不
cloudra/web_launcher.py:352             # 含 cloudra 子目录，是确定无遮蔽的工作目录。
cloudra/web_launcher.py:353             cwd=frontend,
cloudra/web_launcher.py:354         )
cloudra/web_launcher.py:355     try:
cloudra/web_launcher.py:356         _wait_healthy(f"http://{probe_host}:{api_port}/api/v2/runtime/health")
cloudra/web_launcher.py:357         from cloudra.services.api_token import resolve_api_token
cloudra/web_launcher.py:358
cloudra/web_launcher.py:359         token = resolve_api_token()
cloudra/web_launcher.py:360         shown_url = _ui_url(probe_host, api_port, token)
cloudra/web_launcher.py:361         print(f"[cloudra] web UI: {shown_url}", flush=True)
```

```
        if bind_host in _WILDCARD_BIND_HOSTS:
            # 通配绑定常常是「服务在远端、浏览器在别处」（SSH/局域网）：
            # 控制台里的回环 URL 在那台机器上打不开，把对外地址一并给出。
            reachable = _primary_address()
            if reachable:
                print(
                    f"[cloudra] reachable from other machines: "
                    f"{_ui_url(reachable, api_port, token)}",
                    flush=True,
                )
        if os.environ.get("CLOUDRA_NO_BROWSER", "").strip().lower() not in {"1", "true", "yes"}:
            webbrowser.open(shown_url)
        if runtime is None:
            return 0
        return runtime.wait()
    except KeyboardInterrupt:
        return 0
    finally:
        if runtime is not None:
            _terminate_runtime(runtime)
```